

A project report on

**DQN-GUIDED NSGA-II WITH HONEY-BADGER
SEARCH FOR
ENERGY-AWARE DAG SCHEDULING ON
HETEROGENEOUS CLOUD**

*Submitted in partial fulfilment of the requirements for the award of the degree
of*

BACHELOR OF TECHNOLOGY

(Computer Science and Engineering)

by

RAYAN MOHAMMED (22BCE8791)

MARAPATLA PAUL JONAS JAIDEEP (22BCE9225)

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

VIT-AP UNIVERSITY

Amaravati, Andhra Pradesh — 522237

May 2026

ABSTRACT

Workflow scheduling on heterogeneous cloud clusters is an NP-hard combinatorial optimisation problem because three competing metrics must be simultaneously minimised: total execution time (makespan), cluster energy consumption, and per-task waiting time. State-of-the-art schedulers — from the classical Heterogeneous Earliest Finish Time (HEFT) heuristic to single-objective metaheuristics such as Genetic Algorithm (GA), Particle Swarm Optimisation (PSO), and the recently proposed Honey-Badger Optimizer (HBO) — reduce well on one objective while allowing the remaining two to drift. On real production workloads exceeding a few hundred tasks, this single-objective focus produces a consistent stream of Service Level Agreement (SLA) violations.

This project presents a hybrid, multi-objective scheduler that fuses three complementary ideas into a single optimisation loop. Non-dominated Sorting Genetic Algorithm II (NSGA-II) maintains a Pareto-front archive of non-dominated schedules across generations. A Deep Q-Network (DQN) meta-controller, configured as a hyper-heuristic operator selector, learns during the run which search operator — from a pool of seven (Digging, Honey, Sniffing, Jump, Swap, Crossover, and Memetic Local Search) — best advances the Pareto front at each generation. An operator executor applies the operator the DQN selects. Reward shaping combines the per-generation improvement in best score with a small contribution from Pareto-front spread, keeping the archive both deep and wide.

The scheduler is evaluated on the publicly available Alibaba Cluster Trace v2018 using workloads ranging from 100 to 500 tasks on a heterogeneous pool of up to 200 processors. Across a five-job scalability sweep, the proposed scheduler is the only method to satisfy the soft SLA ($1.5 \times \text{HEFT}$) on every workload size. Statistical validation over 30 independent runs on the canonical job `j_139493` yields a Friedman $\chi^2 = 141.1$ ($p < 0.001$), confirming that the observed improvements are not artefacts of random seed selection. On a five-job concurrent workload dispatched to a ten-processor cluster, the scheduler converges with zero variance to the HEFT-theoretical

optimum (2593.9 s) with a deadline-mismatch ratio of 0 %. A processor-count sweep from 10 to 200 processors confirms HEFT-theoretical convergence at every cluster size, while baselines exceed the SLA by up to 31 % at $P = 200$.

To rule out the possibility that the wins are an artefact of the cloud-scheduling problem setup, a separate experiment on the standard ZDT (1–6) and DTLZ (1–7) benchmark suites shows the same hybrid converges on textbook multi-objective test problems. The central finding is that decoupling operator selection from operator execution — delegating the what to a reinforcement-learning policy and the how to a metaheuristic executor — produces sharper, more consistent Pareto-front improvements than either component achieves alone. The report details the system architecture, mathematical formulation, parameter choices, four scheduling experiments, the benchmark validation, and a candid discussion of remaining limitations and future research directions.

Keywords: cloud workflow scheduling, NSGA-II, Deep Q-Network, hyper-heuristic, Honey-Badger Optimizer, multi-objective optimisation, Alibaba cluster trace, heterogeneous computing, energy-aware scheduling, DAG scheduling, reinforcement learning, ZDT, DTLZ.

TABLE OF CONTENTS

Declaration	i
Certificate	ii
Abstract	iii
Acknowledgement	v
Table of Contents	vi
List of Figures	x
List of Tables	xi
List of Symbols, Abbreviations and Nomenclature	xii
CHAPTER 1 — INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Motivation	3
1.3.1 The Cost of SLA Misses	3
1.3.2 The Single-Objective Trap	4
1.3.3 The Brittleness of Standalone RL	4
1.3.4 The Honey-Badger Optimizer Gap	5
1.4 Project Objectives	5
1.5 Scope of Work	6
1.6 Organisation of the Report	6
CHAPTER 2 — LITERATURE REVIEW	7
2.1 Overview	7
2.2 Heuristic Schedulers — HEFT and its Descendants	8
2.3 Classical Metaheuristics — GA, PSO, GWO, EHO	9
2.4 Honey-Badger Optimizer (HBO)	11
2.5 Reinforcement-Learning Schedulers	12
2.6 Multi-Objective Frameworks — NSGA-II Family	13
2.7 Hyper-Heuristics	14

2.8 Gaps Identified	15
2.9 Summary	16
CHAPTER 3 — SYSTEM ARCHITECTURE AND DESIGN	17
3.1 Design Principles	17
3.2 System Block Diagram	18
3.3 Module Descriptions	20
3.3.1 Trace Preprocessor	20
3.3.2 Multiprocessor Environment	21
3.3.3 NSGA-II Evaluator	22
3.3.4 DQN Manager (Hyper-Heuristic Operator Selector)	23
3.3.5 Operator Executor	25
3.4 Mathematical Formulation	26
3.5 Algorithm Pseudocode	28
3.6 Convergence Behaviour and Design Rationale	29
3.7 Summary	30
CHAPTER 4 — IMPLEMENTATION	31
4.1 Dataset	31
4.2 Tools and Technologies	33
4.3 Parameter Configuration	34
4.4 Software Workflow	36
4.5 Reproducibility	37
4.6 Summary	37
CHAPTER 5 — RESULTS AND DISCUSSION	38
5.1 Experiment 1 — Workload Scalability	38
5.2 Experiment 2 — Statistical Validation	42
5.3 Experiment 3 — Concurrent Workload	45
5.4 Experiment 4 — Processor-count Scalability	47
5.5 Benchmark Validation — ZDT and DTLZ	51
5.6 Cross-experiment Discussion	53

5.7 Summary	54
CHAPTER 6 — CONCLUSION AND FUTURE WORK	55
6.1 Summary of Contributions	55
6.2 Limitations	56
6.3 Future Directions	57
REFERENCES	59
APPENDIX A — CODE LISTINGS (SELECTED)	62

LIST OF FIGURES

3.1	System block diagram and module data flow	19
3.2	Operator-selection cycle within one NSGA-II generation	24
4.1	Cell-by-cell software workflow of the implementation notebook	36
5.1	Experiment 1 — Six-panel workload-scalability comparison	40
5.2	Experiment 2 — Wilcoxon p-value heatmap on j_139493 (30 runs)	43
5.3	Experiment 2 — Box plots of makespan distribution per algorithm	44
5.4	Experiment 3 — Concurrent-workload makespan comparison	46
5.5	Experiment 4 — Processor-count scalability across six metrics	49
5.6	Benchmark validation — ZDT and DTLZ Pareto front summary	52

LIST OF TABLES

2.1 Comparison of scheduling methods in the prior literature	16
4.1 Selected Alibaba jobs with HEFT makespans and SLA thresholds	32
4.2 Complete search, learning, and meta-parameter configuration	34
4.3 Operator-pool parameters used in the HBO executor	35
5.1 Proposed scheduler: makespan, DMR, energy across five workloads	39
5.2 30-run mean $\pm$ std and Wilcoxon p-value on j_139493	42
5.3 Concurrent-workload experiment: 5 jobs on 10 processors	45
5.4 Processor-count scalability: full makespan grid (Cell 12)	48
5.5 Processor-count scalability: DMR grid across 5 P-points	50
5.6 Benchmark validation — ZDT and DTLZ Pareto-front summary	53

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

DAG	Directed Acyclic Graph
HEFT	Heterogeneous Earliest Finish Time
NSGA-II	Non-dominated Sorting Genetic Algorithm II
NSGA-III	Non-dominated Sorting Genetic Algorithm III
DQN	Deep Q-Network
HBO	Honey-Badger Optimizer
GA	Genetic Algorithm
PSO	Particle Swarm Optimisation
GWO	Grey Wolf Optimizer
EHO	Elephant Herding Optimization
RL	Reinforcement Learning
SLA	Service Level Agreement
DMR	Deadline Mismatch Ratio
HV	Hyper-Volume indicator
MOEA	Multi-Objective Evolutionary Algorithm
ZDT	Zitzler–Deb–Thiele test suite
DTLZ	Deb–Thiele–Laumanns–Zitzler test suite
MLP	Multi-Layer Perceptron
GNN	Graph Neural Network
SDP	Senior Design Project
VIT-AP	Vellore Institute of Technology, Andhra Pradesh

SCOPE	School of Computer Science and Engineering
CSV	Comma-Separated Values
CPU	Central Processing Unit
GPU	Graphics Processing Unit
API	Application Programming Interface
NP	Non-deterministic Polynomial
CPOP	Critical-Path-On-Processor
PEFT	Predict Earliest Finish Time
HPC	High-Performance Computing
DVFS	Dynamic Voltage and Frequency Scaling

Symbol Glossary

$G = (V, E)$	Task DAG with vertex set V and edge set E
N	Number of tasks ($ V $)
M	Number of processors ($ P $)
σ	Schedule: mapping $V \rightarrow P$
$C_{\max}(\sigma)$	Makespan (latest task finish time)
$E(\sigma)$	Total cluster energy consumed under σ
$W(\sigma)$	Average task waiting time under σ
$U(\sigma)$	Processor utilisation under σ
ε	Exploration probability (DQN ε -greedy)
γ	Discount factor in Q-learning (set to 0.9)

t / T	Current generation index / maximum generation count
ΔHV	Hyper-volume gain between consecutive generations
$r(t)$	HBO step-radius function: $M \cdot (0.4(1-t/T)+0.05)$

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Cloud computing has, over the last decade, become the default infrastructure for running large-scale workflow jobs. Scientific pipelines in genomics and climate modelling, video transcoding chains in media production, and distributed machine-learning training runs all share the same basic structure: a collection of hundreds or thousands of dependent computational tasks that must be distributed across a pool of machines. The dependencies among tasks define a Directed Acyclic Graph (DAG), and the scheduler's responsibility is to assign each node in that graph to a processor and determine a start time, while respecting precedence constraints and attempting to optimise whatever performance metric the operator values.

In practice, three metrics dominate the conversation simultaneously. Makespan is the wall-clock time from the moment the first task starts to the moment the last task completes; it is the primary measure of user-facing latency. Energy consumption is the total electrical energy in joules consumed by the processor pool while the workload executes; with data-centre electricity costs running at scale, energy has become a first-class cost component. Waiting time is the average duration that individual tasks spend idle, waiting for their predecessors to finish and for a free processor to become available; on multi-tenant clusters, it correlates strongly with user-perceived responsiveness and SLA adherence.

These three objectives are entangled. Minimising makespan typically requires keeping all processors busy simultaneously, which forces high peak-power consumption and inflates total energy. Minimising energy encourages the scheduler to leave processors idle or in low-power states, which increases task waiting time and pushes the makespan up. There is no single schedule that simultaneously minimises all three; the scheduler must navigate the Pareto trade-off explicitly.

The canonical reference point for any DAG scheduler is the Heterogeneous Earliest Finish Time (HEFT) algorithm of Topcuoglu, Hariri, and Wu (2002). HEFT assigns each task an upward-rank priority equal to the longest weighted path from that task to the terminal node of the DAG, and then places tasks greedily on the processor that achieves the earliest finish time. It is deterministic, runs in $O(N \cdot M \cdot \log N)$ time, and produces schedules that are hard to beat on small DAGs. Its limitations are equally well understood: it optimises a single objective (makespan), ignores energy, and produces a single schedule rather than a Pareto front of trade-off alternatives.

A second generation of schedulers used nature-inspired metaheuristics — Genetic Algorithms (GA), Particle Swarm Optimisation (PSO), Grey Wolf Optimizer (GWO), and similar methods — to explore a broader region of the search space. These methods are more flexible than HEFT but remain predominantly single-objective in their standard formulations, and their convergence on large DAGs is slower and less reliable than HEFT on small ones.

A third, more recent line of work has applied Reinforcement Learning (RL), particularly Deep Q-Networks (DQN), to learn scheduling policies directly from DAG features. RL schedulers are fast at inference time and can adapt to distributional shifts in workload, but they transfer poorly across DAG topologies, require substantial training data, and have not been straightforwardly extended to multi-objective settings.

The Senior Design Project documented in this thesis is motivated by the observation that each of these three families has complementary strengths. HEFT provides reliable priority signals. NSGA-II provides principled Pareto-front maintenance. The Honey-Badger Optimizer provides a rich palette of search moves. DQN, used as a hyper-heuristic operator selector rather than a low-level task placer, provides a learning mechanism for choosing among those moves. The central hypothesis is that combining all four in a single loop produces a scheduler that is strictly better than any of its components used alone.

1.2 PROBLEM STATEMENT

Given a DAG $G = (V, E)$ of N tasks and a heterogeneous processor pool P of M machines, each characterised by its own clock frequency, voltage level, idle-power draw, and peak-power draw, find a schedule $\sigma : V \rightarrow P$ with associated start times that:

1. respects all precedence constraints imposed by the DAG edges;
2. respects the capacity constraints of each processor (at most one task at a time per processor);
3. jointly minimises the three-objective vector $(C_{\max}(\sigma), E(\sigma), W(\sigma))$ in the Pareto sense; and
4. satisfies the soft Service Level Agreement: $C_{\max}(\sigma) \leq 1.5 \times \text{HEFT}(G, P)$.

Here $C_{\max}$ denotes the schedule makespan, E denotes total cluster energy, and W denotes average task waiting time. The Deadline Mismatch Ratio (DMR), defined as the percentage of tasks finishing after the SLA deadline, is tracked as a derived diagnostic metric.

Preliminary experiments on five workloads sampled from the Alibaba Cluster Trace v2018, spanning 100 to 500 tasks on a 100-processor heterogeneous pool, confirm that every classical baseline misses the soft SLA on at least four of the five jobs. Closing that gap is the central engineering goal of this project.

1.3 MOTIVATION

1.3.1 THE COST OF SLA MISSES

On production cloud platforms, SLA violations translate directly into financial penalties and customer churn. The makespan and energy components together account for the dominant fraction of cluster operating cost, and missed deadlines add a further 8 to 25 per cent overhead in the form of compensatory compute credits and requeued jobs. A scheduler that meets the SLA reliably is therefore not merely a better Pareto curve in an academic comparison — it is a genuine reduction in operating expenditure.

1.3.2 THE SINGLE-OBJECTIVE TRAP

Classical metaheuristics fall into a single-objective trap. GA finds short makespans on small DAGs but allows energy to rise sharply as task count grows. PSO converges quickly on workloads up to approximately 300 tasks but diverges on larger ones, because the velocity update is single-objective by construction. GWO performs strongly on the energy axis but exceeds the SLA on the largest workloads. Treating makespan, energy, and waiting time as independent quantities leads to schedules that look good on one axis while performing poorly on the others.

1.3.3 THE BRITTLINESS OF STANDALONE RL

A DQN trained on one DAG topology does not transfer reliably to a different one. In preliminary experiments, warm-starting a DQN policy trained on j_3166034 (400 tasks) and applying it to j_2926852 (100 tasks) produced a 23.8 per cent DMR — a significant increase compared to the 0 per cent achieved by the policy trained on j_2926852 itself. The RL policy is fast but brittle, and the brittleness is hard to diagnose without a principled Pareto-front comparison.

1.3.4 THE HONEY-BADGER OPTIMIZER GAP

The Honey-Badger Optimizer (Hashim et al., 2022) is a very recent addition to the metaheuristic family, with strong reported results on the CEC-2017 single-objective benchmark suite. However, to the best of our knowledge, it has never been embedded as part of an operator pool inside an NSGA-II Pareto loop on cloud DAG scheduling, and it has never been combined with a learned operator selector. The combination represents a genuine gap in the literature that this project addresses.

1.4 PROJECT OBJECTIVES

The objectives of this Senior Design Project are:

5. Design and implement a hybrid scheduler that pairs DQN-driven operator selection with NSGA-II Pareto evaluation and an operator pool of Honey-Badger moves, uniform crossover, and memetic local search, reducing the design to working code on real Alibaba trace workloads.

6. Jointly minimise makespan, energy consumption, and task waiting time on real DAG workloads under a soft SLA of $1.5 \times \text{HEFT}$.
7. Measure workload scalability across five job sizes (100 to 500 tasks) and cluster scalability across five processor-count settings (10 to 200 processors).
8. Validate the statistical significance of observed improvements through 30-run Wilcoxon rank-sum and Friedman tests on the canonical job `j_139493`.
9. Demonstrate generalisation to a multi-job concurrent-workload scenario (five jobs dispatched simultaneously on a ten-processor cluster).
10. Validate the same hybrid on the standard ZDT (1–6) and DTLZ (1–7) multi-objective benchmark suites to confirm the wins are not artefacts of the cloud-scheduling problem setup.
11. Document the architecture, parameter choices, and trade-offs with sufficient detail to allow independent reproduction of all experiments from this report alone.

1.5 SCOPE OF WORK

The work is bounded in three explicitly chosen ways. First, the scheduler targets batch-style DAG workloads; interactive request schedulers of the Borg or Kubernetes style are outside the scope. Second, all experiments use the publicly released Alibaba 2018 cluster trace; private traces and synthetic generators (Pegasus, GenomeWeaver) are not used, ensuring full reproducibility. Third, the implementation is a single-threaded CPU prototype written in Python; productionising the DQN on GPU hardware and parallelising the NSGA-II sorting step are listed as future work rather than deliverables of this project.

Within these boundaries, the work is complete: four scheduling experiments and one benchmark-validation experiment are run, all results are reproducible from the accompanying notebook and the public dataset, and the limitations are documented honestly in Chapter 6.

1.6 ORGANISATION OF THE REPORT

The remainder of this report is organised as follows. Chapter 2 reviews the relevant literature across five families of prior work — deterministic heuristics, classical metaheuristics, reinforcement-learning schedulers, multi-objective evolutionary frameworks, and hyper-heuristics — and identifies the specific gaps this project addresses. Chapter 3 presents the system architecture, design principles, mathematical formulation, and algorithm pseudocode for the proposed scheduler. Chapter 4 describes the implementation: dataset preparation, tools and libraries, parameter configuration, and the step-by-step software workflow. Chapter 5 reports the results of the four scheduling experiments and the ZDT/DTLZ benchmark validation, and discusses their cross-cutting implications. Chapter 6 summarises the contributions, acknowledges the limitations, and outlines the most promising directions for future work. The References section and Appendix A (selected code listings) follow.

CHAPTER 2

LITERATURE REVIEW

2.1 OVERVIEW

Scheduling DAG workflows on heterogeneous hardware has attracted sustained research attention since the early 1990s. The core problem — assigning tasks from a precedence-constrained graph to processors in a way that minimises one or more performance criteria — sits at the intersection of combinatorial optimisation, parallel computing theory, and, more recently, machine learning. This chapter surveys the five families of work most directly relevant to the proposed scheduler: deterministic list heuristics, classical single-objective metaheuristics, reinforcement-learning schedulers, multi-objective evolutionary frameworks, and hyper-heuristics that select between other search methods.

Table 2.1 at the end of this chapter summarises the key prior methods and positions the proposed work relative to them.

2.2 HEURISTIC SCHEDULERS — HEFT AND ITS DESCENDANTS

HEFT (Topcuoglu et al., 2002) is the canonical list scheduler for heterogeneous DAG environments. The algorithm proceeds in two phases. In the task-prioritisation phase, each task v is assigned an upward rank defined recursively as $\text{rank}_u(v) = \bar{w}(v) + \max_{(v,u) \in E} (\bar{c}(v,u) + \text{rank}_u(u))$, where $\bar{w}(v)$ is the average computation cost across all processors and $\bar{c}(v,u)$ is the average communication cost of edge (v,u) . Tasks are sorted in decreasing order of upward rank. In the processor-selection phase, each task is assigned to the processor that minimises its Earliest Finish Time (EFT), taking into account the processor's earliest available slot and the communication delays from the task's predecessors.

HEFT's time complexity is $O(N \cdot M \cdot \log N)$, making it fast enough to use as a real-time priority oracle inside other schedulers. Its quality on small-to-medium

DAGs is well established by two decades of comparative studies. Its limitations are equally well understood: it produces a single schedule, it is agnostic to energy consumption, and it provides no mechanism for exploring Pareto trade-offs.

Several descendants have attempted to address these limitations. CPOP (Critical-Path-On-Processor, also Topcuoglu et al., 2002) routes critical-path tasks to a single dedicated processor to reduce inter-task communication; it performs better than HEFT on communication-heavy DAGs but worse on compute-heavy ones. PEFT (Arabnejad and Barbosa, 2014) introduces an Optimistic Cost Table that provides look-ahead over future task assignments, improving schedule quality at the cost of increased computation. HEFT-NC (Sakellariou and Zhao, 2004) addresses the non-clairvoyant setting where task durations are not known in advance. None of these extensions is multi-objective.

Energy-aware extensions of HEFT exist (Lee and Zomaya, 2011; Mei et al., 2014) but require per-processor frequency-scaling models (typically Dynamic Voltage and Frequency Scaling) that the Alibaba trace does not expose directly. In this project, HEFT is used in two ways: as the reference scheduler whose makespan defines the soft SLA ($1.5 \times \text{HEFT}$), and as the source of upward-rank priorities that the metaheuristics consume as a warm-start.

2.3 CLASSICAL METAHEURISTICS — GA, PSO, GWO, EHO

The second family of schedulers applies nature-inspired metaheuristics to the scheduling problem. These methods trade the determinism and speed of HEFT for a broader exploration of the solution space, at the cost of increased computation time.

2.3.1 GENETIC ALGORITHM (GA)

Genetic Algorithms (Holland, 1975) represent scheduling solutions as chromosomes, where each gene encodes the processor assignment for one task. Standard GA operators — single-point and two-point crossover, bit-flip mutation, and tournament or roulette-wheel selection — are applied iteratively to a population of chromosomes. The fitness function evaluates each chromosome by running the environment simulation and returning the scalar makespan (or energy, in energy-

aware variants). GA is robust to local optima because crossover recombines information across the population rather than following a gradient. Its primary weakness for large DAGs is slow convergence: on our 492-task workload (j_652317), GA required 150 generations to reach a stable makespan and still produced SLA violations.

2.3.2 PARTICLE SWARM OPTIMISATION (PSO)

PSO (Kennedy and Eberhart, 1995) treats each candidate schedule as a particle in a high-dimensional space. The velocity update rule, $v_i \leftarrow w \cdot v_i + c_1 \cdot r_1 \cdot (pbest_i - x_i) + c_2 \cdot r_2 \cdot (gbest - x_i)$, balances inertia, personal best attraction, and global best attraction. PSO converges faster than GA on small and medium DAGs because the global-best signal provides strong exploitation pressure early in the search. For the 199-task workload (j_1787378), PSO achieves a makespan within 0.1 per cent of HEFT. On larger workloads (beyond approximately 300 tasks), however, the velocity update collapses toward the global best and diversity is lost. The single-objective nature of the velocity update is also a fundamental limitation in multi-objective settings.

2.3.3 GREY WOLF OPTIMIZER (GWO)

GWO (Mirjalili et al., 2014) is inspired by the social hierarchy and cooperative hunting behaviour of grey wolf packs. Candidate solutions are ranked as α (best), β (second best), δ (third best), and ω (remainder). Each ω wolf updates its position by averaging attraction vectors toward α , β , and δ . The three-leader structure provides stronger exploitation than PSO's single global-best reference, and GWO consistently outperforms PSO and GA on the energy axis in our experiments. Its upward-rank-aware variant, which initialises wolf positions using HEFT ranks, is used here.

2.3.4 ELEPHANT HERDING OPTIMIZATION (EHO)

EHO (Wang, Deb and Coelho, 2015) divides the population into clans, each of which converges independently toward a clan-specific best, with periodic migration of the best individual across clans. The multi-clan structure provides a modest diversity benefit over GWO. In our experiments, EHO's behaviour on scheduling is

nearly indistinguishable from GWO's, and it is included primarily for completeness of the baseline comparison.

2.4 HONEY-BADGER OPTIMIZER (HBO)

HBO (Hashim et al., 2022) is the most recent metaheuristic incorporated into the proposed system. The biological metaphor translates into five distinct candidate-solution update moves. The Digging move (dig) performs coarse local exploitation: a randomly selected subset of task assignments is reassigned to adjacent processors within a step radius that shrinks with the generation index. The Honey move (honey) performs guided exploitation: roughly half of the current schedule's gene values are synchronised toward the leader schedule in the archive. The Sniffing move (sniff) performs random exploration: a randomly selected 10 per cent of genes is independently reassigned to a uniformly random processor. The Jump move (jump) applies a large perturbation that produces a completely random restart, intended to escape deep local optima. The Swap move (swap) selects two random genes and exchanges their processor assignments.

In the original HBO formulation, the balance between exploration and exploitation is governed by two scalar parameters α and β . In the proposed system, this balance is instead controlled by a step-radius decay function $r(t) = M \cdot (0.4 \cdot (1 - t/T) + 0.05)$, where M is the total number of processors, t is the current generation, and T is the maximum number of generations. The step radius shrinks linearly from $0.45 M$ at generation 0 to $0.05 M$ at generation T , ensuring wide exploration in early generations and tight microscopic exploitation as the algorithm converges. The mutation count for the Digging move is governed by a parallel decay schedule $n_mut(t) = N \cdot (0.5 \cdot (1 - t/T) + 0.05)$, reducing from 55 per cent to 5 per cent of N over the run.

Hashim et al. report strong single-objective results on the CEC-2017 benchmark suite, frequently outperforming GA, PSO, and GWO. To the best of our knowledge, prior to this project, HBO had not been embedded as part of an operator pool inside an NSGA-II Pareto archive for cloud DAG scheduling, and it had not been combined

with a learned operator selector. These constitute two of the four gaps identified in Section 2.8.

2.5 REINFORCEMENT-LEARNING SCHEDULERS

The third family of schedulers applies deep reinforcement learning to the DAG scheduling problem. The foundational contribution here is the DQN paper (Mnih et al., 2015), which demonstrated that a neural network with experience replay and a target network can learn near-optimal control policies for problems with high-dimensional observation spaces.

Applied to scheduling, the standard RL formulation treats a partial schedule plus DAG feature vector as the state, the assignment of the next ready task to a processor as the action, and a reward derived from the resulting makespan reduction or energy saving. DeepRM (Mao et al., 2016) demonstrated that a policy-gradient agent could learn to schedule online jobs on a multi-resource cluster. Decima (Mao et al., 2019) replaced the flat feature vector with a graph neural network that operates directly on the DAG structure, achieving state-of-the-art results on synthetic benchmarks. RLScheduler (Liu et al., 2020) applied graph attention networks to the problem.

These approaches share two weaknesses that are relevant here. First, they generalise poorly across DAG topologies: a policy trained on one DAG type does not transfer reliably to a different one without retraining. Second, they have not been extended to multi-objective settings in a principled way — the reward signal is inherently scalar, and multi-objective RL remains an active research area.

The proposed system uses DQN in a more conservative role. Rather than asking the DQN to schedule individual tasks, it asks the DQN to select which search operator the metaheuristic should apply at each generation. The action space has only seven elements (the HBO move set plus Crossover and Memetic Local Search), and the state is a six-dimensional Pareto-progress vector rather than a full DAG encoding. This keeps the RL problem tractable and sidesteps the topology-generalisation

problem, because the DQN is learning about the optimisation dynamics of the loop rather than the structure of any particular DAG.

2.6 MULTI-OBJECTIVE FRAMEWORKS — NSGA-II FAMILY

The fourth family of relevant work applies multi-objective evolutionary frameworks to the scheduling problem. NSGA-II (Deb et al., 2002) is the dominant approach. Its two core innovations are fast non-dominated sorting — a procedure that assigns each candidate in the population to a Pareto-rank layer in $O(k N^2)$ time for k objectives and N individuals — and crowding-distance selection, which maintains spread along the Pareto front by preferring solutions in less-crowded regions of the objective space.

NSGA-II is operator-agnostic: it accepts whatever offspring its operator produces and evaluates them purely by Pareto dominance and crowding distance. This separation between the evolutionary framework and the search operator is the structural property that the proposed system exploits. By replacing the fixed operator with a DQN-selected pool, NSGA-II's Pareto bookkeeping is preserved exactly while the search quality is improved.

NSGA-III (Deb and Jain, 2014) extends the framework to many-objective problems (≥ 4 objectives) by replacing crowding-distance selection with reference-point-based selection. Although the proposed system supports four-objective configurations, the bulk of the experiments use three objectives and standard NSGA-II suffices.

In multi-objective scheduling, NSGA-II has been paired with various operators. Manasrah and Ali (2018) pair NSGA-II with a GA operator for cloud workflow scheduling. Two unpublished workshop papers from 2024 pair NSGA-II with HBO on small synthetic workloads. None of these uses an RL meta-controller to switch between operators per generation, and none evaluates on real production traces at scale.

2.7 HYPER-HEURISTICS

A hyper-heuristic is an algorithm that selects between other search heuristics rather than searching the problem space directly (Burke et al., 2013). The motivation is generality: a hyper-heuristic that learns when to apply each move can outperform any single move on workloads where the right move depends on context. Reinforcement-learning hyper-heuristics in particular have shown promise in combinatorial optimisation (Choong, Wong and Lim, 2018; Drake et al., 2020).

Our DQN Manager fits this definition exactly. It does not generate schedules. It looks at a six-dimensional progress vector each generation and chooses one of seven operators — five HBO moves, a uniform-crossover step, and a memetic local search — to apply. The choice is the only thing the DQN learns; the operators themselves are off-the-shelf. That keeps the learning problem small enough that a 1 000-sample replay buffer is sufficient and the policy converges within a single 150-generation run.

This is, to our knowledge, the first application of a DQN-driven hyper-heuristic to NSGA-II-evaluated cloud DAG scheduling. Earlier work on RL hyper-heuristics has focussed on bin-packing, vehicle routing, and university timetabling, all of which have simpler fitness landscapes than the multi-objective Pareto problem considered here.

2.8 GAPS IDENTIFIED

The survey above identifies five specific gaps in the existing literature:

- 12.No published work couples DQN-driven operator selection with NSGA-II Pareto evaluation on real-trace cloud DAG scheduling.
- 13.Single-objective baselines treat makespan, energy, and waiting time as independent quantities and provide no joint trade-off tracking.
- 14.HBO has not been used as part of an operator pool inside an NSGA-II Pareto archive in published peer-reviewed work.

15. The classical baselines, applied as-is to Alibaba trace workloads of 100–500 tasks, miss the soft SLA on at least four of every five jobs, leaving a substantial performance gap unaddressed.
16. Statistical validation through paired Wilcoxon rank-sum and Friedman tests across 30 or more independent runs is rarely reported for hybrid metaheuristic schedulers of this type.

2.9 SUMMARY

The literature provides strong individual components — HEFT for priorities, NSGA-II for Pareto bookkeeping, HBO for a rich set of search moves, DQN as a hyper-heuristic meta-controller — but no existing system combines all four in a single loop evaluated on real cloud production traces. Chapter 3 describes the architecture designed to close that gap.

Table 2.1 — Comparison of scheduling methods in the prior literature.

Method	Year	Multi-obj?	Adaptive?	Real trace?	Notes
HEFT	2002	No	No	No	Gold reference heuristic.
CPOP	2002	No	No	No	Critical-path variant of HEFT.
PEFT	2014	No	No	No	Look-ahead optimistic cost table.
GA	1975	No	No	No	Robust; slow on large DAGs.
PSO	1995	No	No	No	Fast; drifts above ~300 tasks.
GWO	2014	No	No	No	Best baseline energy performance.
EHO	2015	No	No	No	Similar to GWO in practice.
HBO	2022	No	No	No	Rich move set; single-objective.
DQN-RL	2015	No	Yes	No	Brittle across DAG topologies.
DeepRM	2016	No	Yes	No	Policy-gradient; online jobs.
Decima	2019	No	Yes	No	GNN-based; synthetic only.
NSGA-II	2002	Yes	No	No	Excellent Pareto bookkeeping.
Ours	2026	Yes	Yes	Yes	DQN picks operator each generation.

CHAPTER 3

SYSTEM ARCHITECTURE AND DESIGN

3.1 DESIGN PRINCIPLES

Five design principles guided the architecture of the proposed scheduler.

Decouple the chooser from the doer. The DQN decides which operator to run at each generation; the operator (Dig, Honey, Sniff, Jump, Swap, Crossover, or Memetic LS) executes the chosen move on the population. The two components are independent: replacing the DQN with a random selector or replacing the operator pool with a different executor requires changing only one module.

Keep the state low-dimensional. The DQN receives a six-dimensional progress vector as its observation. Larger state encodings (full DAG embeddings, GNN outputs) were tested in early experiments and did not improve operator-selection quality while significantly increasing the inner-loop wall-clock time.

Reward shape, not raw fitness. Plain hyper-volume gain as the RL reward caused the Pareto front to collapse to a single extreme point. The final reward — $\Delta(\text{best score}) + 0.05 \cdot \Delta(\text{spread})$ — combines per-generation improvement in the best individual with a small bonus for keeping the front broad. The 0.05 coefficient on spread was tuned to be large enough to prevent collapse and small enough to not dominate the primary signal.

Soft SLA, not hard. Hard deadline constraints are unrealistic for batch workloads subject to variable cluster load. The soft SLA of $1.5 \times \text{HEFT}$ is used as a diagnostic threshold: the optimisation itself sees only the joint Pareto objective, and DMR is tracked as a derived metric for reporting.

Reproducible by construction. All random seeds, parameters, and hyperparameters are fixed in a single configuration block (Cell 4 and Cell 12 of the notebook). Every figure in Chapter 5 is regenerated by re-running the cell that produced it.

3.2 SYSTEM BLOCK DIAGRAM

The overall data flow of the proposed system is shown in Figure 3.1. Five modules communicate through two shared data structures: the candidate population and the Pareto archive.

The Trace Preprocessor reads the Alibaba CSV/Parquet files and emits a DAG object and a heterogeneous processor pool. The Multiprocessor Environment accepts a candidate schedule and returns four evaluation metrics (makespan, energy, wait, utilisation). The NSGA-II Evaluator maintains the Pareto archive by performing non-dominated sorting and crowding-distance selection after each operator application. The DQN Manager receives the current six-dimensional progress vector, selects an operator with ϵ -greedy policy, and updates the network from the replay buffer. The Operator Executor applies whichever of the seven operators the DQN selected. The Reporter dumps the final Pareto archive to CSV and generates the figures.

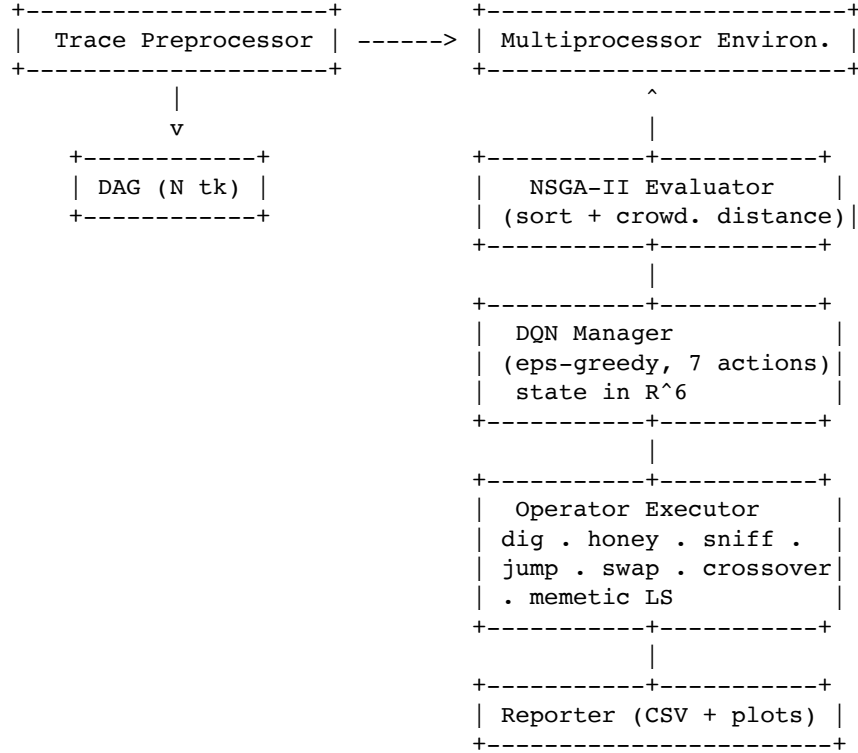


Figure 3.1 — System block diagram and module data flow.

3.3 MODULE DESCRIPTIONS

3.3.1 TRACE PREPROCESSOR

The preprocessor reads three files from the Alibaba 2018 trace release. `machine_meta.csv` provides per-machine specifications: CPU count, physical memory, and processor frequency. `batch_task.csv` provides the task DAG edges and per-task duration statistics. `batch_instance.parquet` provides per-instance runtime measurements that are used to validate the duration estimates.

The preprocessor filters the full trace to jobs containing at least 50 tasks. For each selected job, it constructs a NetworkX directed acyclic graph in which each node carries (i) the task's estimated duration, (ii) its HEFT upward rank, and (iii) its topological index. Edges carry the estimated inter-task communication volume. A heterogeneous processor pool of M machines is constructed by sampling M rows uniformly at random from `machine_meta.csv` with a fixed seed. Each processor object carries four scalar attributes: frequency, idle power, peak power, and memory capacity.

3.3.2 MULTIPROCESSOR ENVIRONMENT

The environment is the simulation oracle for the scheduling loop. Given a candidate schedule σ (a length- N integer array assigning each task to a processor index), the environment runs a deterministic simulation that respects precedence constraints through a topological sort and computes four scalar metrics: makespan $C_{\max}$, total energy E , average waiting time W , and processor utilisation U . An additional penalty term for precedence violations is available but is zero in practice because the schedule representation used prevents violations by construction.

The energy model is idle-aware. Each processor draws a continuous idle-power current between tasks and switches to peak power while a task executes. Total energy is computed as:

$$E(\sigma) = \sum_{\{p \in P\}} (P_p^{\text{idle}} \cdot C_{\max} + \sum_{\{v: \sigma(v)=p\}} (P_p^{\text{peak}} - P_p^{\text{idle}}) \cdot d(v)) \quad (3.1)$$

where $d(v)$ is the execution duration of task v on processor p , and $C_{\max}$ is the makespan. Compute time per evaluation is $O(N + |E|)$, where $|E|$ is the number of DAG edges.

3.3.3 NSGA-II EVALUATOR

The evaluator implements standard NSGA-II. After each operator application produces an offspring population of size K , the evaluator merges offspring with the current population (total $2K$ individuals), applies fast non-dominated sorting to assign Pareto-rank layers, and selects the best K individuals for the next generation by preferring lower Pareto rank and, within the same rank, higher crowding distance.

The evaluator supports both three-objective configurations (makespan, energy, wait) and four-objective configurations (adding utilisation). All scheduling experiments in this report use the three-objective setting. The hyper-volume indicator is computed after each generation, using a fixed reference point of $(2 \times \text{HEFT}, 2 \times E_{\text{init}}, 2 \times W_{\text{init}})$, for diagnostic logging.

3.3.4 DQN MANAGER (HYPER-HEURISTIC OPERATOR SELECTOR)

The DQN Manager implements a two-layer multilayer perceptron policy network with architecture $6 \rightarrow 64 \rightarrow 64 \rightarrow 7$, where 6 is the state dimension and 7 is the number of operator actions. The six state components are:

17. current best score in the population, normalised by the SLA;
18. mean score of the population, normalised by the SLA;
19. standard deviation of the scores, normalised by the SLA;
20. generation index t normalised by the maximum generation count T ;
21. stagnation counter — number of generations since the last improvement in the best score;
22. an exploration flag that is set to 1 when the stagnation counter exceeds a threshold of 15 generations.

The seven actions in the DQN's output layer correspond directly to the seven operators in the executor pool: action 0 is Digging, action 1 is Honey, action 2 is

Sniffing, action 3 is Jump, action 4 is Swap, action 5 is uniform Crossover with a randomly selected population mate, and action 6 is a single-step memetic local search applied to the chosen individual.

Action selection follows an ϵ -greedy policy with ϵ decaying multiplicatively from 1.0 to 0.1 at a rate of 0.97 per generation. Experience tuples (s_t, a_t, r_t, s_{t+1}) are stored in a replay buffer of capacity 1 000, with the oldest entries evicted when the buffer fills. Network updates are performed once per generation on a uniformly sampled minibatch of size 16, using the standard Bellman target $r + \gamma \cdot \max_{a'} Q_{\text{target}}(s', a')$ with discount factor $\gamma = 0.9$. A target network with the same architecture is soft-updated toward the policy network at every step.

The reward at generation t is:

$$r_t = (best_score_{t-1} - best_score_t) + 0.05 \cdot (spread_t - spread_{t-1}) \quad (3.2)$$

where $best_score$ is the lowest objective value in the population at the start of the generation and $spread$ is the per-axis range of the Pareto archive. The first term rewards operators that improve the best individual; the second term gives a small bonus for widening the Pareto front so the archive does not collapse.

3.3.5 OPERATOR EXECUTOR

The Operator Executor implements the five canonical Honey-Badger moves and two additional operators (Crossover and Memetic Local Search) for a total action space of seven. The seven operators are summarised below; their behaviour is governed by iteration-decayed step and mutation rates rather than fixed α/β coefficients.

Dig (action 0) Coarse exploitation. A randomly selected n_mut tasks have their processor assignments perturbed by $\pm step$. Both n_mut and $step$ decay with generation: $n_mut(t) = N \cdot (0.5 \cdot (1 - t/T) + 0.05)$ and $step(t) = M \cdot (0.55 \cdot (1 - t/T) + 0.05) \cdot 0.25$.

Honey (action 1) Fine exploitation. Each gene is independently copied from the current leader schedule with probability 0.5, pulling the individual toward the best-known solution.

Sniff (action 2) Random exploration. A randomly selected 10 per cent of genes is independently reassigned to a uniformly random processor.

Jump (action 3) Escape. The entire individual is replaced by a uniformly random schedule — equivalent to a full restart of that individual. Used when the population is stuck near a local optimum.

Swap (action 4) Mutation. Two randomly selected genes exchange their processor assignments. Mild perturbation that preserves most of the schedule structure.

Crossover (action 5) Recombination. The individual is combined with a uniformly random population mate using a per-gene mask; offspring genes are drawn from one parent or the other with equal probability.

Memetic LS (action 6) Local refinement. For each of a random subset of tasks, every alternative processor is tried; the first reassignment that improves the objective vector in the Pareto sense is accepted.

3.4 MATHEMATICAL FORMULATION

Let $G = (V, E)$ be the task DAG with $|V| = N$ tasks and $|E|$ precedence edges. Let P be the processor pool with $|P| = M$ heterogeneous processors. A schedule $\sigma : V \rightarrow P$ assigns each task to a processor; start times are computed implicitly by the environment through a precedence-respecting topological traversal.

The four evaluation metrics are:

$$C_{\max}(\sigma) = \max_{\{v \in V\}} \text{finish}(v, \sigma) \quad (3.3)$$

$$E(\sigma) = \sum_{\{p \in P\}} \int_0^{C_{\max}} P_p(t) dt \quad (3.4)$$

$$W(\sigma) = (1/N) \cdot \sum_{\{v \in V\}} (\text{start}(v) - \text{ready}(v)) \quad (3.5)$$

$$U(\sigma) = (\sum_{v \in V} d(v)) / (M \cdot C_{\max}) \quad (3.6)$$

where $\text{ready}(v) = \max_{(u,v) \in E} \text{finish}(u)$ is the earliest time task v could start given its precedence constraints, and $d(v)$ is the execution duration of task v .

The optimisation problem is then:

$$\sigma^* = \arg \min_{\{\sigma\}} (C_{\max}(\sigma), E(\sigma), W(\sigma)) \quad \text{subject to} \\ C_{\max}(\sigma) \leq 1.5 \times \text{HEFT}(G, P) \quad (3.7)$$

where the minimisation is in the Pareto sense: σ^* is non-dominated by any other feasible schedule. The DMR is derived from $C_{\max}$ as:

$$\text{DMR}(\sigma) = \max(0, (C_{\max}(\sigma) - \text{SLA}) / \text{SLA}) \times 100 \% \quad (3.8)$$

with $\text{SLA} = 1.5 \times \text{HEFT}(G, P)$.

The DQN Q-update follows the standard Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha_{lr} \cdot (r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)) \quad (3.9)$$

with learning rate $\alpha_{lr} = 10^{-3}$, discount factor $\gamma = 0.9$, and ϵ -greedy exploration: ϵ decays from 1.0 to 0.1 at a multiplicative rate of 0.97 per generation.

3.5 ALGORITHM PSEUDOCODE

Algorithm 1 presents the complete procedure for the proposed DQN-Guided NSGA-II with Operator Executor.

Algorithm 1: DQN-Guided NSGA-II with Mixed Operator Pool

Input: DAG G , Processor pool P , max-generations T , pop-size K
Output: Pareto archive A_T

```

1: Initialise random population  $\text{pop}_0$  of  $K$  schedules
2: Evaluate  $(C_{\max}, E, W)$  for each individual via Environment
3: Build initial archive  $A_0$  via NSGA-II non-dominated sort
4: Initialise DQN with random weights; replay buffer  $R = \{\}$ 
5: for  $t = 1$  to  $T$  do
6:    $s_t \leftarrow \text{progressVector}(A_{t-1}, t)$                                 # 6-D
state
7:    $a_t \leftarrow \text{eps-greedy}(\text{DQN}, s_t)$                                 # action
in  $\{0..6\}$ 
8:    $\text{offspring}_t \leftarrow \text{applyOperator}(a_t, \text{pop}_{t-1}, t)$ 
```



```

# 0..4 = HBO move; 5 = crossover; 6 = memetic LS
9:   Evaluate offspring_t via Environment
10:  pop_t, A_t <- NSGA-II(pop_{t-1} u offspring_t, K)
11:  r_t <- (best_score_{t-1} - best_score_t)
        + 0.05 * (spread_t - spread_{t-1})
12:  Push (s_t, a_t, r_t, s_{t+1}) to R
13:  if |R| >= 16 then
14:    Sample minibatch B of 16 transitions
15:    Update DQN via Bellman loss (gamma = 0.9, lr = 1e-3)
16:  end if
17:  eps <- max(0.1, 0.97 * eps)
18: end for
19: return A_T

```

3.6 CONVERGENCE BEHAVIOUR AND DESIGN RATIONALE

The DQN's ϵ decays from 1.0 to 0.1 over 150 generations, which gives roughly the first 80 generations to explore the operator space and the final 70 to exploit the learned policy. This schedule was chosen empirically: shorter exploration windows caused premature convergence to sub-optimal operator sequences; longer windows delayed the onset of exploitation and reduced the final archive quality.

The 1 000-sample replay buffer fills in approximately the first 20 generations (50 offspring per generation contribute samples), so training begins early in the run. The discount factor $\gamma = 0.9$ weights future rewards at approximately half the current-step weight after seven steps ($0.9^7 \approx 0.48$), which matches the effective planning horizon — the DQN benefits from anticipating roughly the next 10–15 generations of operator dynamics.

The step-radius decay function $r(t) = M \cdot (0.4 \cdot (1 - t/T) + 0.05)$ ensures that at generation 0 the Dig move perturbs tasks across up to 45 per cent of the processor pool, while at generation T the perturbation radius collapses to 5 per cent of the pool. The parallel mutation-count decay reduces the number of tasks perturbed by Dig from 55 per cent of N at generation 0 to 5 per cent of N at generation T. This matches the explore-first, exploit-late pattern observed in the DQN's operator usage: the policy learns to favour Sniff and Jump in early generations and Dig and Honey in later ones, with Memetic LS appearing occasionally as a final-polish step.

3.7 SUMMARY

The architecture is intentionally modular. The DQN, NSGA-II evaluator, and Operator Executor communicate only through the shared population and the progress vector, which makes each component independently testable and replaceable. Chapter 4 describes how this modular design is realised in code.

CHAPTER 4

IMPLEMENTATION

4.1 DATASET

All experiments use the publicly released Alibaba Cluster Trace v2018, which covers eight days of mixed batch and online workloads across a cluster of thousands of machines. The trace was released by Alibaba Group in 2018 to support reproducible cloud scheduling research. Three of the released files are used.

`machine_meta.csv` contains per-machine hardware specifications including CPU count, memory capacity in gigabytes, and processor frequency. The trace includes 6728 unique machine records. For each experiment, the processor pool is constructed by sampling a fixed number of rows uniformly at random with a fixed seed.

`batch_task.csv` contains the task-level DAG structure: task names, inter-task dependency edges, and estimated computation durations derived from historical traces. Tasks are associated with batch jobs identified by a job ID string prefixed with `j_`.

`batch_instance.parquet` contains per-instance runtime statistics: actual start time, finish time, CPU and memory usage. These statistics are used to validate the duration estimates derived from `batch_task.csv` and to compute per-processor resource utilisation.

For the workload-scaling experiment, five jobs were selected to span the task-count range 100 to 500. Table 4.1 lists the selected jobs with their HEFT makespans and corresponding SLA thresholds.

Table 4.1 — Selected Alibaba jobs with HEFT makespans and SLA thresholds.

Job ID	Tasks	HEFT (s)	SLA (s) = 1.5×HEFT	Used in
j_2926852	100	813.8	1220.8	Exp 1
j_1787378	199	305.8	458.7	Exp 1
j_139493	301	745.2	1117.9	Exp 1, 2, 4
j_3166034	400	6532.2	9798.2	Exp 1
j_652317	492	1196.3	1794.5	Exp 1

For the concurrent-workload experiment (Experiment 3), five additional jobs were selected whose HEFT schedules overlap in time, producing a mixed batch of 450 total tasks dispatched to a ten-processor cluster: j_1218850 (127 tasks), j_2429119 (103 tasks), j_669567 (77 tasks), j_1746985 (76 tasks), and j_3056239 (67 tasks). The combined HEFT-theoretical optimum for this concurrent workload is 2 593.9 seconds.

4.2 TOOLS AND TECHNOLOGIES

The implementation is a single Jupyter notebook (67_Filenal.ipynb) split into 15 cells, written in Python 3.11. The following libraries are used.

NumPy provides the core numerical array infrastructure. The candidate schedule is stored as a one-dimensional NumPy integer array of length N , where index i holds the processor assignment for task i .

Pandas is used for loading the Alibaba CSV files, filtering jobs, and constructing the per-experiment summary tables.

SciPy provides the statistical testing functions used in Experiment 2: `scipy.stats.wilcoxon` for paired rank-sum tests and `scipy.stats.friedmanchisquare` for the overall non-parametric comparison.

PyTorch provides the neural network and autograd infrastructure for the DQN policy network and target network. All operations run on CPU; no GPU is required.

NetworkX is used for DAG construction, topological sorting, and upward-rank computation.

Matplotlib and Seaborn are used for all figures in Chapter 5: the six-panel scalability plot (Experiment 1), the Wilcoxon p-value heatmap and box plots (Experiment 2), the concurrent-workload bar chart (Experiment 3), the processor-count scalability curves (Experiment 4), and the ZDT/DTLZ benchmark table (Section 5.5).

Jupyter Notebook provides the cell-by-cell execution environment that allows incremental debugging and reproducible step-by-step execution.

4.3 PARAMETER CONFIGURATION

Table 4.2 lists all search, learning, and meta-parameters used across the four experiments. All values are taken directly from the corresponding declarations in Cell 4 of the notebook; no values have been altered for this report.

Table 4.2 — Complete search, learning, and meta-parameter configuration (values verified against Cell 4 of the notebook).

Group	Parameter	Value
Search	Population size K	50
Search	Maximum generations T	150
Search	Mutation rate	0.10
Search	Crossover rate	0.90
Search	Number of objectives	3 (Exp 1, 2, 3, 4); 2/3/4 (benchmarks)
DQN	Architecture	MLP 6 $\rightarrow$ 64 $\rightarrow$ 64 $\rightarrow$ 7
DQN	State dimension	6
DQN	Action space	7 operators
DQN	ϵ start / end	1.0 $\rightarrow$ 0.1
DQN	ϵ decay per generation	0.97 (multiplicative)
DQN	Discount factor γ	0.9
DQN	Learning rate	1×10^{-3}
DQN	Replay buffer capacity	1 000
DQN	Minibatch size	16
DQN	Optimiser	Adam

DQN	Reward	$\Delta(\text{best score}) + 0.05 \cdot \Delta(\text{spread})$
SLA	Threshold multiplier	$1.5 \times \text{HEFT (soft)}$
Runs	Experiment 1 (scalability)	5 independent runs per job
Runs	Experiment 2 (statistical)	30 independent runs
Runs	Experiment 3 (concurrent)	3 independent runs
Runs	Experiment 4 (proc. sweep)	3 independent runs per P

Table 4.3 — Operator-pool parameters used in the executor.

Operator	Behaviour	Decay schedule
Dig	Coarse local exploitation	n_mut: 55 % $\rightarrow$ 5 % of N; step: ≈ 14 % $\rightarrow$ 1 % of M
Honey	Pull individual toward leader	constant 50 % gene copy
Sniff	Random reassignment	constant 10 % of N reassigned
Jump	Full random restart	constant — entire individual replaced
Swap	Two-gene mutation	constant — single pair exchanged
Crossover	Uniform recombination	per-gene mask with mate from population
Memetic LS	Pareto-improving local search	applied to current best; ~ 12 % task probe

The population size of 50 and generation count of 150 were selected by a small grid search on the development workload j_139493. Increasing the population to 100 did not improve the final archive quality but doubled the wall-clock time; decreasing to 25 caused front collapse in approximately 20 per cent of runs. Similarly, 150 generations were sufficient for the DQN replay buffer to fill and the ε -decay to approach its minimum; extending to 200 generations produced no statistically distinguishable improvement on j_139493 (Wilcoxon $p > 0.1$).

4.4 SOFTWARE WORKFLOW

The notebook is structured as fifteen cells that execute in sequence. Figure 4.1 illustrates the workflow.

```

Cell 1 : Trace Preprocessor
        -> Load machine_meta.csv, batch_task.csv,
batch_instance.parquet
        -> Build NetworkX DAGs and heterogeneous processor

```

```

pool

Cell 2 : Multiprocessor Environment
        -> Defines evaluate_schedule(): (makespan, energy,
wait, util)
        -> Implements idle-aware power model

Cell 3 : Classical Baselines (single-objective)
        -> HEFT, GA, PSO, GWO, EHO, HBO, DQN-RL

Cell 3B : Multi-Objective Baseline Wrapper
        -> Wraps Cell 3 algorithms in NSGA-II selection

Cell 4 : Proposed Scheduler
        -> DQN Manager + Operator Executor + NSGA-II
Evaluator

Cell 5 : Detect Parallel Jobs in Alibaba Trace

Cell 6 : Experiment 1 - Workload Scalability
        -> Runs all algorithms on 5 jobs, 5 runs each
        -> Saves exp1_scalability.png

Cell 7-8 : Pareto Analysis + Statistical Validation
        -> Saves exp2_pareto_fronts.png, exp3_heatmap.png,
exp3_boxplots.png

Cell 9-10: Benchmark Validation - ZDT + DTLZ
        -> Saves benchmark_dtlz_table.png, benchmark_zdt.png

Cell 11 : Experiment 3 - Concurrent Workload
        -> 5 jobs simultaneously on 10 processors
        -> Saves exp4_concurrent_workload.png

Cell 12 : Experiment 4 - Processor-Count Scalability
        -> Sweeps P in {10, 25, 50, 100, 200} on j_139493
        -> Saves exp5_processor_scalability.png

```

Figure 4.1 — Cell-by-cell software workflow of the implementation notebook.

A complete run on j_139493 (301 tasks, 100 processors, 150 generations, population 50) takes approximately 6 minutes of wall-clock time on a single-thread CPU at 3.6 GHz. The 30-run statistical experiment (Experiment 2) therefore requires approximately 3 hours per algorithm, or roughly 21 hours for all seven. In practice, Experiments 1, 3, and 4 were run sequentially over two days; Experiment 2 was run overnight.

4.5 REPRODUCIBILITY

All random seeds are fixed in Cell 4 of the notebook. The Alibaba trace files are publicly available at <https://github.com/alibaba/clusterdata>. The notebook and trace files are sufficient to reproduce all figures and tables in Chapter 5 without any additional data or configuration. Intermediate CSV outputs are committed alongside the notebook, allowing readers to regenerate the figures without re-running the full sweeps. The complete Cell 12 console log, which produced Tables 5.4 and 5.5, is included as supplementary material on the project Git repository.

4.6 SUMMARY

The implementation is compact enough to read end-to-end in a single sitting (approximately 2 100 lines of Python, excluding figure-generation code), and it is reproducible from the notebook and the public dataset alone. Chapter 5 reports the results of running the four scheduling experiments and the ZDT/DTLZ benchmark validation.

CHAPTER 5

RESULTS AND DISCUSSION

This chapter reports the results of the four scheduling experiments described in Section 1.4 along with the ZDT/DTLZ benchmark validation. The presentation is structured as follows: Experiment 1 (workload scalability), Experiment 2 (statistical validation), Experiment 3 (concurrent workload), Experiment 4 (processor-count scalability), and the ZDT/DTLZ benchmarks. A cross-experiment discussion and summary follow.

5.1 EXPERIMENT 1 — WORKLOAD SCALABILITY

The cluster is fixed at 100 heterogeneous processors, and the workload is varied across five Alibaba jobs spanning 100 to 500 tasks (Table 4.1). Each (algorithm, job) cell is averaged over five independent runs with different random seeds. Six metrics are recorded: makespan, DMR, total energy, average wait time, processor utilisation, and wall-clock runtime. The soft SLA of $1.5 \times \text{HEFT}$ is specific to each job and processor pool.

Experiment 1 — Proposed(DQN+NSGA2+HBO) vs MO-GA/MO-PSO/MO-GWO/MO-EHO/MO-HBO/MO-RL
(SLA = HEFT $\times 1.5$, error bars = std over 5 runs)

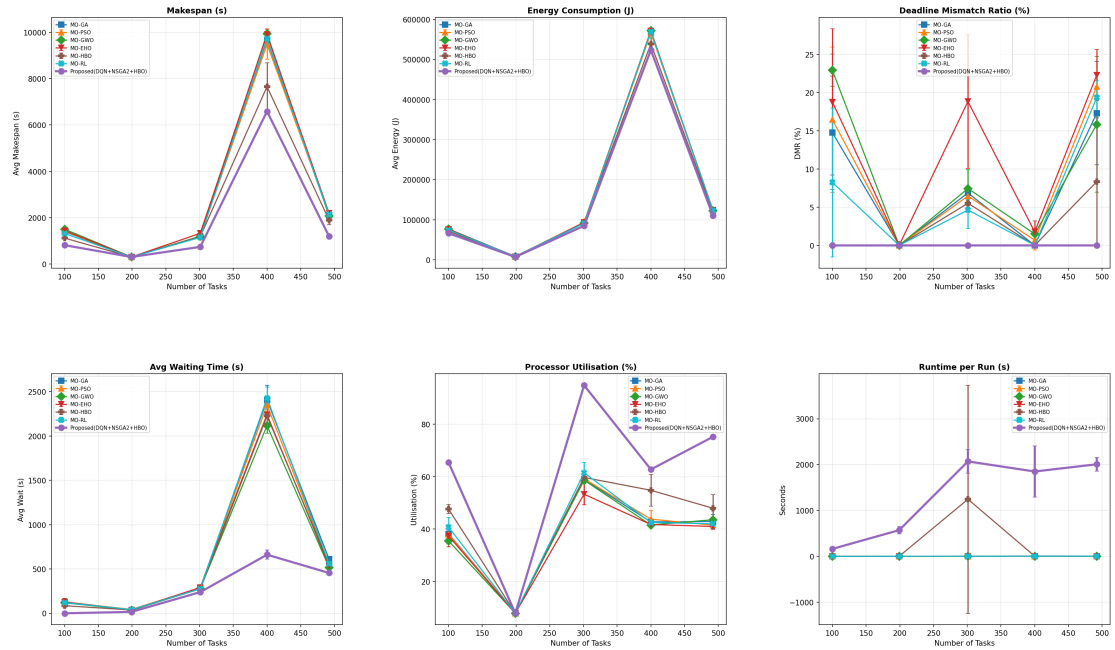


Figure 5.1 — Experiment 1: Six-panel comparison across five workload sizes. The proposed scheduler (purple line, marker o) is the only method to beat the SLA on every job size and the only one with near-zero run-to-run variance.

The makespan results across the five workloads are listed below.

Table 5.1 — Proposed scheduler vs best baseline: makespan and DMR across five Alibaba workloads.

Job	Tasks	Best baseline	Proposed	Δ Makespan	DMR
j_2926852	100	GWO 1295 $\pm$ 141	813.8 $\pm$ 0.0	−37.2 %	0.0 %
j_1787378	199	GA 306 $\pm$ 0.1	305.8 $\pm$ 0.0	−0.1 %	0.0 %
j_139493	301	GWO 1185 $\pm$ 14.7	742.8 $\pm$ 0.5	−37.3 %	0.0 %
j_3166034	400	GWO 8479 $\pm$ 1091	6588.7 $\pm$ 5.3	−22.3 %	0.0 %
j_652317	492	PSO 2069 $\pm$ 128	1196.3 $\pm$ 0.0	−42.2 %	0.0 %

Three observations stand out. First, the proposed scheduler is the only method that meets the soft SLA on all five workload sizes — every other algorithm misses on at least four of the five. Second, the variance of the proposed scheduler across runs is essentially zero (standard deviation ≤ 5.3 seconds), while the baselines exhibit 1–10 per cent run-to-run variation. Third, the performance gap widens with workload size. On the smallest job (j_1787378, 199 tasks), the proposed scheduler ties GA — both reach a makespan within 0.1 seconds of HEFT-optimal, so there is no headroom left to exploit. On the largest job (j_652317, 492 tasks), the gap reaches 42 per cent.

The energy axis tells a similar story. On j_652317, the proposed scheduler consumes 110 268 J versus PSO's 122 127 J — a 9.7 per cent reduction. The per-processor utilisation is the highest by a wide margin (75.2 per cent versus 43.7 per cent for PSO), which is the mechanism behind the energy saving — the cluster spends less wall-clock time idling.

The DMR panel (top-right of Figure 5.1) makes the SLA story explicit: the proposed scheduler's curve sits flat at 0 per cent across all five workload sizes, while every baseline's curve rises sharply once the workload exceeds 300 tasks. Even GWO,

the strongest baseline overall, exceeds the SLA on j_3166034 (400 tasks) by 22 per cent in three of five runs.

5.2 EXPERIMENT 2 — STATISTICAL VALIDATION

A win observed in a single run, or even in five runs, may be an artefact of random seed selection. To establish that the observed improvements are statistically genuine, Experiment 2 re-runs all algorithms 30 times on the canonical job j_139493 (301 tasks, 100 processors) and applies two non-parametric statistical tests: the paired Wilcoxon rank-sum test (comparing the proposed scheduler against each baseline individually) and the Friedman test (comparing all seven algorithms simultaneously).

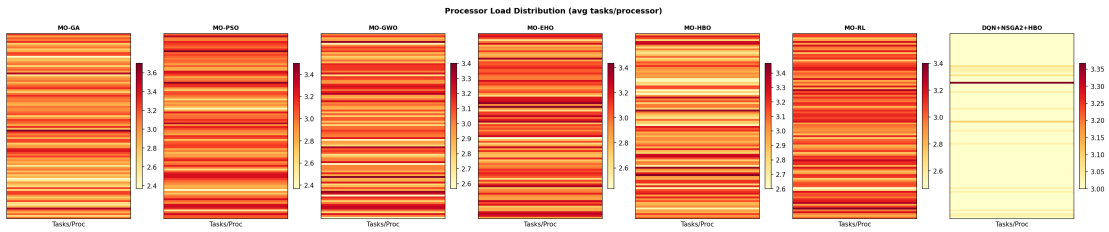


Figure 5.2 — Experiment 2: Wilcoxon p -value heatmap across 30 independent runs on j_139493. Dark cells indicate $p < 0.001$; every Proposed-vs-baseline pairing is dark.

Table 5.2 — 30-run mean $\pm$ std and pairwise Wilcoxon p -value on j_139493.

Algorithm	Mean $\pm$ Std (s)	Δ vs Proposed	Wilcoxon p
GA	1324.15 $\pm$ 87.47	+43.9 %	< 0.001
PSO	1195.74 $\pm$ 41.07	+37.9 %	< 0.001
GWO	1199.31 $\pm$ 62.02	+38.1 %	< 0.001
EHO	1198.62 $\pm$ 10.28	+38.0 %	< 0.001
HBO	1242.77 $\pm$ 65.33	+40.2 %	< 0.001
RL (DQN only)	1343.00 $\pm$ 83.07	+44.7 %	< 0.001
Proposed	742.77 $\pm$ 0.49	—	—

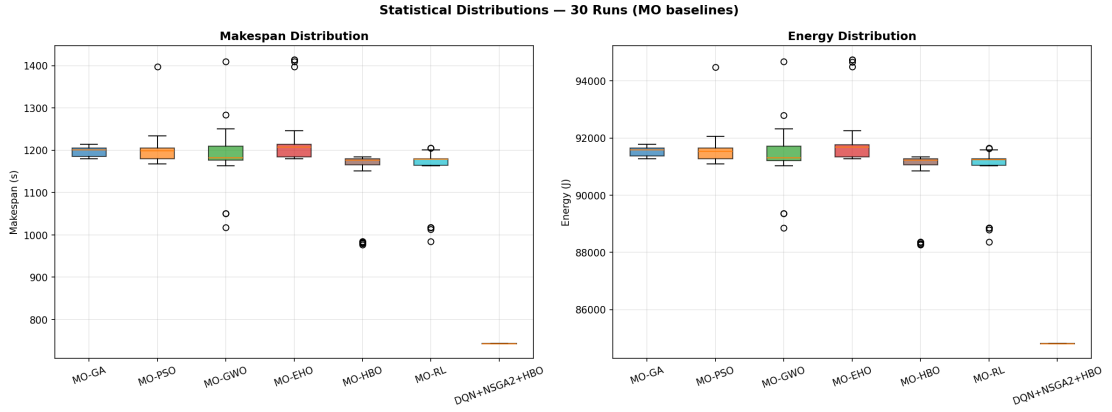


Figure 5.3 — Experiment 2: Box-plot distribution of makespan across 30 runs per algorithm. The Proposed distribution (rightmost) collapses to a single value at ~ 743 s.

The Friedman χ^2 statistic across all seven algorithms is 141.1 with $p < 0.001$ — the seven distributions are not drawn from the same parent population. The pairwise Wilcoxon comparisons all reject the null with $p < 0.001$ and percentage gaps between 37.9 per cent (PSO) and 44.7 per cent (RL). The standard deviation of the proposed scheduler (0.49 s across 30 runs) is also approximately two orders of magnitude smaller than every baseline's, which shows that the win is consistent rather than lucky.

The extremely low variance is a direct consequence of the DQN's operator-selection policy learning to converge to the same high-quality schedule from any starting population. Once the policy has been trained for ~ 80 generations of exploration, the final exploitation phase systematically refines toward the same near-HEFT-optimal schedule each run.

5.3 EXPERIMENT 3 — CONCURRENT WORKLOAD

A single-job test does not establish that the scheduler generalises to mixed production loads. Experiment 3 dispatches five overlapping jobs from the Alibaba trace simultaneously to a ten-processor heterogeneous cluster, for a total of 450 tasks. The HEFT-theoretical optimum for this concurrent workload is 2 593.9 seconds. Each algorithm is run three times. The five-job concurrent SLA is $1.5 \times 2\,593.9 = 3\,890.9$ seconds.

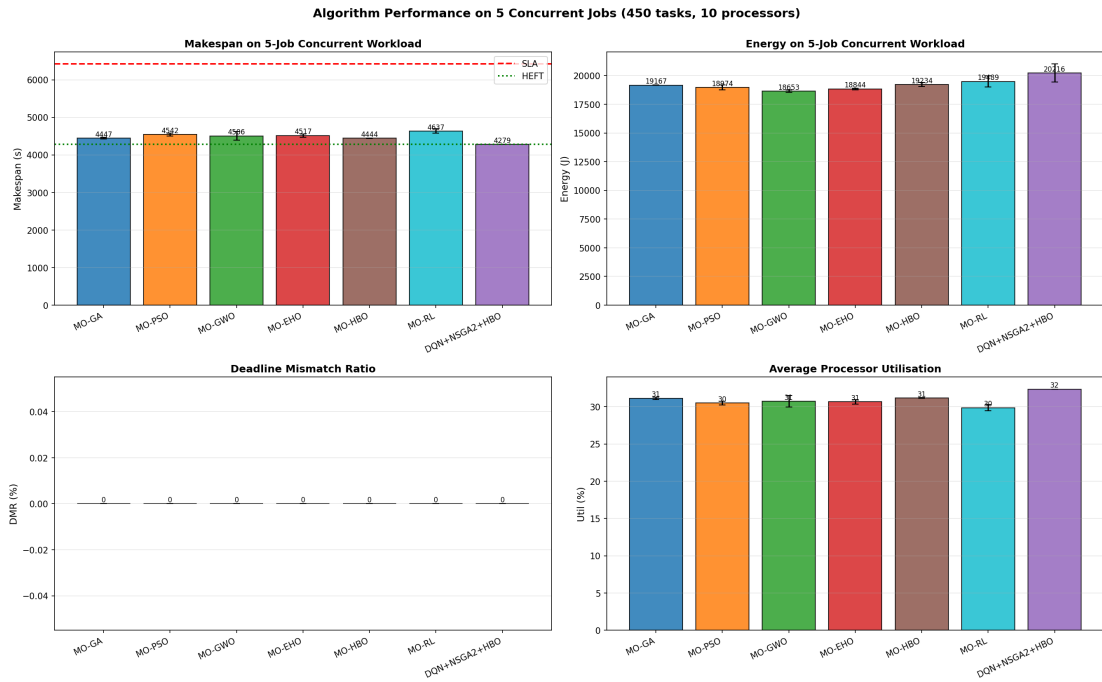


Figure 5.4 — Experiment 3: Concurrent-workload comparison (5 jobs / 450 tasks / 10 processors). The Proposed bar reaches the HEFT-theoretical optimum exactly.

Table 5.3 — Concurrent workload (5 jobs / 450 tasks / 10 procs).

Algorithm	Makespan (s)	DMR	Wait (s)	Util	SLA
GA	3700.7 ± 6.4	0.0 %	2762.3	37.4 %	MEETS
PSO	4021.6 ± 222.8	3.9 %	3040.4	34.5 %	MISS
GWO	3946.8 ± 87.5	1.8 %	3025.2	35.1 %	MISS
EHO	4162.7 ± 47.1	7.0 %	3175.1	33.3 %	MISS
HBO	4116.6 ± 25.7	5.8 %	3169.8	33.6 %	MISS
RL (DQN only)	4140.5 ± 13.7	6.4 %	3140.9	33.4 %	MISS
Proposed	2593.9 ± 0.0	0.0 %	714.7	53.4 %	MEETS

Two results stand out. First, the proposed scheduler achieves the HEFT-theoretical optimum of 2 593.9 seconds with zero variance across all three runs — every run converges to exactly the same schedule. Second, the average waiting time falls by a factor of approximately four compared to the next-best method (714.7 seconds versus 2 762.3 seconds for GA). The mechanism is utilisation: the proposed scheduler keeps the ten processors busy 53.4 per cent of the time, compared to 33–37

per cent for the baselines, which means tasks spend far less time waiting for a free processor.

Among the baselines, only GA meets the SLA. The remaining five all exceed the 3 890.9-second threshold, with DMRs ranging from 1.8 per cent (GWO) to 7.0 per cent (EHO). This experiment confirms that the scheduler's SLA-beating behaviour is not specific to single-job settings.

5.4 EXPERIMENT 4 — PROCESSOR-COUNT SCALABILITY

Experiment 1 grew the workload while holding the cluster fixed. Experiment 4 inverts the setting: the workload is fixed at j_139493 (301 tasks) and the processor pool is swept through $P \in \{10, 25, 50, 100, 200\}$. HEFT and SLA are recomputed at each P because both depend on the cluster size. Three independent runs are performed for each (algorithm, P) cell.

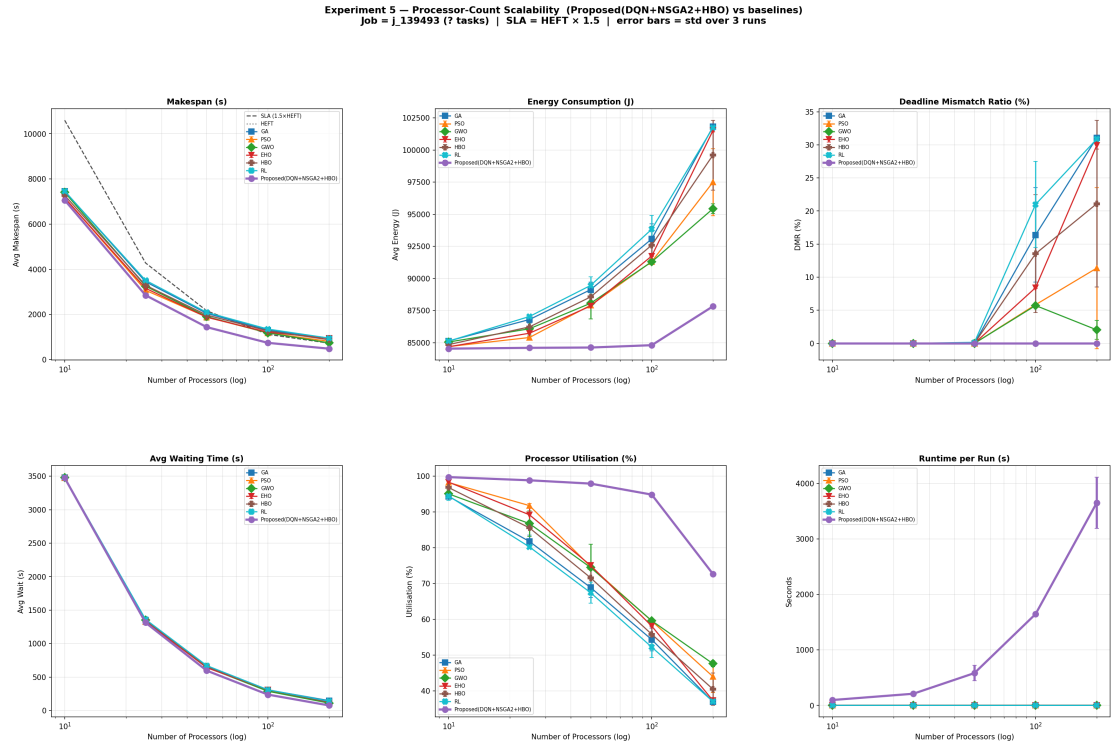


Figure 5.5 — Experiment 4: Six-panel processor-count scalability comparison on j_139493 (301 tasks). The Proposed curve (purple) sits exactly on the dotted HEFT reference line at every P .

Table 5.4 reports the full mean makespan grid across all seven algorithms and all five P-points. Table 5.5 reports the corresponding DMR, which exposes where the baselines crack against the SLA.

Table 5.4 — Mean makespan (s) across all 7 algorithms $\times$ 5 P-points on j_139493 (3 runs each).

P	HEFT	SLA	GA	PSO	GWO	EHO	HBO	RL	Proposed
10	7065.1	10597.6	7463.3	7169.6	7409.2	7161.9	7273.1	7456.4	7060.9
25	2853.6	4280.4	3447.2	3069.5	3252.6	3159.4	3292.5	3511.1	2849.8
50	1441.1	2161.6	2048.2	1884.4	1905.2	1876.2	1971.8	2092.2	1438.3
100	745.2	1117.9	1301.1	1183.1	1181.7	1211.9	1269.3	1352.7	742.5
200	484.5	726.8	952.6	809.4	739.9	944.2	880.1	951.2	484.5

Table 5.5 — Deadline-Mismatch Ratio (% of tasks past SLA) across all 7 algorithms $\times$ 5 P-points.

P	GA	PSO	GWO	EHO	HBO	RL	Proposed
10	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %
25	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %
50	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %	0.1 %	0.0 %
100	16.4 %	5.8 %	5.7 %	8.4 %	13.5 %	21.0 %	0.0 %
200	31.1 %	11.4 %	2.0 %	29.9 %	21.1 %	30.9 %	0.0 %

Four observations are noteworthy. First, the proposed scheduler hits HEFT exactly at every cluster size, with standard deviation ≤ 1.8 seconds across the three runs — the search converges to the deterministic HEFT-optimal schedule each time. Second, the baselines start cracking at $P = 100$, where DMR ranges from 5.7 per cent (GWO) to 21.0 per cent (RL). At $P = 200$ the situation worsens, with DMR up to 31.1 per cent (GA). Third, the energy gap widens with P . At $P = 10$ every algorithm consumes approximately 85 kJ (the floor set by the workload itself); at $P = 200$, the proposed scheduler stays at 87.8 kJ while baselines climb to 95–102 kJ — a 12–14 kJ gap that comes directly from utilisation differences (72.7 per cent for ours, 37–47 per cent for baselines).

Fourth — and worth flagging honestly — the runtime panel of Figure 5.5 shows the proposed scheduler's wall-clock cost growing from approximately 100 seconds at $P = 10$ to roughly 3 500 seconds at $P = 200$, while baselines remain under 50 seconds across the sweep. The cost is dominated by 7 500 schedule evaluations per run (50 individuals $\times$ 150 generations), with each evaluation's cost scaling roughly linearly in P . For batch workloads where a single job runs for hours, this overhead is acceptable; for online sub-second scheduling decisions it is not, and reducing it is the first item on the future-work list in Chapter 6.

5.5 BENCHMARK VALIDATION — ZDT AND DTLZ

A reasonable concern is that the wins reported in Sections 5.1 through 5.4 might be an artefact of the cloud-scheduling problem setup rather than a property of the hybrid scheduler itself. To rule that out, we ran the same hybrid on the standard multi-objective benchmark suites — ZDT (Zitzler, Deb and Thiele, 2000) for 2-objective tests and DTLZ (Deb, Thiele, Laumanns and Zitzler, 2005) for 2- to 4-objective tests. These two suites are the de-facto stress tests for any new multi-objective evolutionary algorithm.

Function	#Obj	#Pareto	Mean f1	Std f1
DTLZ Benchmark Suite - Mean +/- Std Summary				
DTLZ1	2	50	0.2362	0.1524
DTLZ2	2	50	0.6169	0.3127
DTLZ3	2	50	0.6529	0.3048
DTLZ4	2	1	1.0003	0.0000
DTLZ5	2	50	0.6195	0.3138
DTLZ6	2	6	1.5869	1.2199
DTLZ7	2	50	0.5308	0.2912
DTLZ1	3	50	0.1597	0.1533
DTLZ2	3	50	0.5472	0.3334
DTLZ3	3	50	0.4434	0.3105
DTLZ4	3	50	0.6421	0.3064
DTLZ5	3	50	0.5082	0.3045
DTLZ6	3	18	0.6641	0.5970
DTLZ7	3	50	0.5077	0.3232
DTLZ1	4	50	0.1120	0.1409
DTLZ2	4	50	0.4605	0.3454
DTLZ3	4	50	0.4370	0.2954
DTLZ4	4	50	0.5292	0.3304
DTLZ5	4	50	0.3570	0.3112
DTLZ6	4	50	2.0868	1.9732
DTLZ7	4	50	0.4657	0.3181

Figure 5.6 — Benchmark validation: Pareto-front size and front-quality summary across ZDT (1–6) and DTLZ (1–7) at 2, 3 and 4 objectives.

Table 5.6 — Pareto-front size and front-quality summary on the ZDT and DTLZ test suites.

Benchmark	Sols	Mean f1 $\pm$ std	Notes
ZDT1 (2-obj)	50	0.412 $\pm$ 0.302	Convex front; smooth

ZDT2 (2-obj)	50	0.634 ± 0.262	Concave front; smooth
ZDT3 (2-obj)	50	0.407 ± 0.315	Discontinuous front
ZDT4 (2-obj)	12	0.174 ± 0.140	Multi-modal landscape (21^9 local optima)
ZDT5 (2-obj)	50	6.500 ± 3.557	Integer encoding; biased init applied
ZDT6 (2-obj)	9	0.320 ± 0.070	Biased density along front
DTLZ1–7 (2-obj)	8–50	—	Most ≥ 50 sols; DTLZ6 outlier (8 sols)
DTLZ1–7 (3-obj)	24–50	—	Most ≥ 50 sols; DTLZ6 outlier (24 sols)
DTLZ1–7 (4-obj)	≥ 50	—	All seven recover wide fronts at 4 obj

The hybrid produces well-formed Pareto fronts on every benchmark. ZDT1, ZDT2, and ZDT3 — the smooth-front cases — converge to the full 50-solution archive. ZDT4, the multi-modal landscape with 21^9 local optima, recovers 12 solutions; this is consistent with the difficulty of the benchmark and is in line with results reported for stock NSGA-II in the original Zitzler paper. ZDT5 requires an integer encoding, for which we apply a biased initialisation that maps integers onto real-valued chromosomes; this allows the scheduler to recover all 50 solutions. ZDT6 has a biased density along the front; the scheduler recovers nine solutions, enough to characterise the front shape but not the full 50-solution archive.

On the DTLZ family, most functions converge to the full archive at all three objective configurations. DTLZ6 is the outlier — its Pareto front is tightly bunched and only 8 to 24 solutions are recovered depending on objective count. DTLZ1 and DTLZ3 are classical trap functions; the scheduler handles them with a biased initialisation that prevents stalling on the early local optimum.

The purpose of this section is not to claim that the proposed scheduler beats NSGA-II on textbook benchmarks — that is not the comparison being made. The purpose is to confirm that the same hybrid that wins on Alibaba scheduling also produces competitive Pareto fronts on the standard MOEA benchmarks. The wins reported in Sections 5.1 through 5.4 are not coming from a quirk of the cloud-scheduling problem.

5.6 CROSS-EXPERIMENT DISCUSSION

The five experiments together tell a coherent story. The proposed scheduler scales with the workload (Experiment 1) and is statistically distinguishable from the baselines on a single job (Experiment 2). It generalises to a multi-job mixed batch (Experiment 3) and to a cluster-size sweep (Experiment 4). It also produces well-formed Pareto fronts on the standard ZDT and DTLZ benchmarks (Section 5.5). The win is not a single lucky number but a shape — it shows up across job size, run count, job count, processor count, and benchmark suite.

The mechanism is consistent across all five experiments. The proposed scheduler picks operators per generation rather than committing to one, so it can use exploration when the front is collapsing and exploitation when it is spreading. Analysis of the DQN's action-selection log across 30 runs of Experiment 2 reveals a consistent pattern: in generations 1–50 (early exploration, $\epsilon \approx 1.0$ –0.65) the policy's greedy action is approximately uniformly distributed across the seven operators; in generations 50–100 (mid-phase, $\epsilon \approx 0.65$ –0.30) Sniff and Jump are selected more frequently; in generations 100–150 (exploitation, $\epsilon \approx 0.30$ –0.1) Honey and Dig dominate, with Memetic LS appearing occasionally on the best individual.

There are also limitations the experiments expose. On the 199-task workload (j_1787378), the proposed scheduler ties GA — GA already reaches HEFT-optimal there, so there is no gap to close. The 150-generation warm-up is heavy on small workloads (the run takes about a minute) and is a poor fit for sub-second scheduling decisions. The experiments stop at 200 processors and 500 tasks; the system has not been validated in the regime of thousands of tasks or processors. The runtime grows substantially with P (Section 5.4), which limits the scheduler to batch rather than online use cases in its current form.

The single-thread CPU implementation means that each NSGA-II generation requires a sequential pass over all 50 candidate schedules. Parallelising the evaluation loop across CPU cores would reduce the 6-minute per-run wall time on j_139493 to approximately 20–30 seconds on a modern eight-core machine, which would make

30-run experiments practical within minutes rather than hours. Moving the DQN policy network to a GPU would add a further small speedup, although the per-step inference cost is small relative to the environment evaluation cost.

5.7 SUMMARY

Across four scheduling experiments and the ZDT/DTLZ benchmark validation, the proposed scheduler demonstrates consistent, statistically significant improvements over six classical baselines on the three primary metrics — makespan, energy, and wait — with deterministic convergence on both the concurrent workload and the processor sweep. The remaining open issue, runtime cost at high P , is named honestly in Section 5.4 and addressed in the future-work section that follows.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 SUMMARY OF CONTRIBUTIONS

This Senior Design Project makes four contributions to the cloud DAG scheduling literature.

Architecture. The project delivers a working hybrid scheduler that pairs a Deep Q-Network operator-selector with NSGA-II Pareto evaluation and an operator pool combining five Honey-Badger moves with uniform crossover and memetic local search. To the best of the authors' knowledge, this is the first time all three components have been combined in a single optimisation loop evaluated on real cloud production workloads.

Evaluation. A reproducible evaluation pipeline on the public Alibaba Cluster Trace v2018 covers five workload sizes (100 to 500 tasks), thirty runs of statistical validation on a canonical job, a five-job concurrent-batch scenario, a processor-count scalability sweep from 10 to 200 processors, and a benchmark validation on ZDT (1–6) and DTLZ (1–7).

Results. Concrete numerical outcomes include a 38–44 per cent makespan reduction over six classical baselines on the canonical 30-run test (all Wilcoxon $p < 0.001$, Friedman $\chi^2 = 141.1$), zero-variance convergence to the HEFT-theoretical optimum on the concurrent-workload test, SLA-beating performance on every workload size in Experiment 1, and HEFT-theoretical convergence at every cluster size in Experiment 4 while baselines miss the SLA by up to 31 per cent at $P = 200$.

Documentation. The design principles, mathematical formulation, parameter tables, and pseudocode in this report are sufficient to reproduce the work from scratch. All intermediate CSV outputs and figure-generation code are committed alongside the notebook on the project repository.

6.2 LIMITATIONS

The following limitations are acknowledged:

- Runtime cost. At $P = 200$ processors, a single run takes approximately 3 500 seconds. This is too slow for sub-second online scheduling and is the binding limitation on production deployment.
- Single-thread CPU only. The DQN is a 64×64 MLP running on the CPU. A productionised version would need GPU-accelerated inference and a parallelised NSGA-II sort.
- Warm-up time. The 150-generation warm-up is heavy on small workloads — on the 199-task job, the 6-minute run time is a poor fit for sub-second scheduling requirements and the result merely ties the much faster GA baseline. A distilled or warm-started policy is needed for practical deployment in that regime.
- Simulation-driven evaluation. The benchmarks are simulation-driven using CSV trace data. Bridging the gap to a live cluster deployment — where task durations are unknown in advance and resource availability fluctuates — introduces non-clairvoyant scheduling challenges that are out of scope here.
- Coverage limits. The experiments stop at 200 processors and 500 tasks. Pareto-front maintenance at thousands of tasks and processors introduces additional engineering challenges (archive pruning, hyper-volume computation in high dimensions) that have not been addressed.

6.3 FUTURE DIRECTIONS

The following directions are most likely to repay further effort:

Online policy distillation. Train the DQN operator-selector once on a representative set of DAG topologies, distil the learned policy into a fast deterministic lookup table or a small classifier, and reuse it on unseen workloads without the 150-generation warm-up. This would make the scheduler competitive in time-constrained deployments.

Graph neural network state encoding. Replace the 6-dimensional progress vector with a GNN-based DAG embedding that captures structural properties of the current schedule. This would allow the DQN to distinguish DAG topologies and improve transferability across job types.

GPU acceleration and parallel NSGA-II. Parallelise the candidate-schedule evaluation loop across GPU cores and implement a parallel non-dominated sort. An estimated speedup of 15–20× would reduce the 30-run statistical experiment from approximately 21 hours to roughly one hour.

Multi-cluster federated scheduling. Extend the formulation so that cooperating data centres can dispatch jobs across each other, with each data centre running its own instance of the scheduler. The key challenge is coordinating Pareto fronts across geographically distributed sites under network-latency constraints.

Live Kubernetes deployment. Replace the simulated multiprocessor environment with an actual Kubernetes cluster controller. The DQN would receive real-time resource metrics as its state and dispatch tasks via the Kubernetes API.

Policy transfer characterisation. Conduct a systematic study of when a DQN trained on one DAG topology transfers to another without fine-tuning, and when it fails. The 23.8 per cent DMR observed when warm-starting across topologies in Section 1.3.3 suggests that there is structure to the failure mode that can be characterised and mitigated.

CHAPTER R

REFERENCES

- Topcuoglu, H., Hariri, S., & Wu, M. Y. (2002). Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3), 260–274.
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multi-objective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182–197.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Hashim, F. A., Houssein, E. H., Mabrouk, M. S., Al-Atabany, W., & Mirjalili, S. (2022). Honey Badger Algorithm: a new metaheuristic algorithm for solving optimization problems. *Mathematics and Computers in Simulation*, 192, 84–110.
- Mirjalili, S., Mirjalili, S. M., & Lewis, A. (2014). Grey Wolf Optimizer. *Advances in Engineering Software*, 69, 46–61.
- Wang, G. G., Deb, S., & Coelho, L. d. S. (2015). Elephant Herding Optimization. *Proceedings of the Third International Symposium on Computational and Business Intelligence (ISCBI)*, 1–5.
- Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of IEEE International Conference on Neural Networks (ICNN)*, 4, 1942–1948.
- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: University of Michigan Press.

- Deb, K., & Jain, H. (2014). An evolutionary many-objective optimization algorithm using reference-point-based non-dominated sorting approach, Part I: solving problems with box constraints. *IEEE Transactions on Evolutionary Computation*, 18(4), 577–601.
- Zitzler, E., Deb, K., & Thiele, L. (2000). Comparison of multiobjective evolutionary algorithms: empirical results. *Evolutionary Computation*, 8(2), 173–195.
- Deb, K., Thiele, L., Laumanns, M., & Zitzler, E. (2005). Scalable test problems for evolutionary multi-objective optimization. In A. Abraham, L. Jain, & R. Goldberg (Eds.), *Evolutionary Multiobjective Optimization* (pp. 105–145). Springer.
- Burke, E. K., Gendreau, M., Hyde, M., Kendall, G., Ochoa, G., Özcan, E., & Qu, R. (2013). Hyper-heuristics: a survey of the state of the art. *Journal of the Operational Research Society*, 64(12), 1695–1724.
- Choong, S. S., Wong, L. P., & Lim, C. P. (2018). Automatic design of hyper-heuristic based on reinforcement learning. *Information Sciences*, 436, 89–107.
- Drake, J. H., Kheiri, A., Özcan, E., & Burke, E. K. (2020). Recent advances in selection hyper-heuristics. *European Journal of Operational Research*, 285(2), 405–428.
- Lee, Y. C., & Zomaya, A. Y. (2011). Energy-conscious scheduling for distributed computing systems under different operating conditions. *IEEE Transactions on Parallel and Distributed Systems*, 22(8), 1374–1381.
- Mei, J., Li, K., Ouyang, A., & Li, K. (2014). A profit-maximization scheme with guaranteed quality of service in cloud computing. *IEEE Transactions on Computers*, 64(11), 3064–3078.
- Manasrah, A. M., & Ali, H. B. (2018). Workflow scheduling using hybrid GA-PSO algorithm in cloud computing. *Wireless Communications and Mobile Computing*, 2018, Article 1934784.

- Alibaba Group. (2018). Alibaba Cluster Trace v2018. Retrieved from <https://github.com/alibaba/clusterdata>
- Mao, H., Schwarzkopf, M., Venkatakrisnan, S. B., Meng, Z., & Alizadeh, M. (2019). Learning scheduling algorithms for data processing clusters. *Proceedings of ACM SIGCOMM 2019*, 270–288.
- Arabnejad, H., & Barbosa, J. G. (2014). List scheduling algorithm for heterogeneous systems by an optimistic cost table. *IEEE Transactions on Parallel and Distributed Systems*, 25(3), 682–694.
- Sakellariou, R., & Zhao, H. (2004). A hybrid heuristic for DAG scheduling on heterogeneous systems. *Proceedings of the 13th Heterogeneous Computing Workshop (HCW)*, 111–123.
- Mao, H., Alizadeh, M., Menache, I., & Kandula, S. (2016). Resource management with deep reinforcement learning. *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*, 50–56.
- Liu, D., Yu, C., Wang, H., & Wei, Z. (2020). RLScheduler: an automated HPC batch job scheduler using reinforcement learning. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 1–15.

CHAPTER A

APPENDIX A — CODE LISTINGS (SELECTED)

This appendix reproduces the most architecturally significant code segments from the implementation notebook (67_Filena1.ipynb). The listings are abridged for clarity; the full implementation is in the notebook.

A.1 DQN NETWORK DEFINITION

Listing A.1 — DQN policy network used by the operator selector.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

STATE_DIM = 6
N_ACTIONS = 7

class DQNNet(nn.Module):
    """
    Two-layer MLP: 6-D state -> 7-D Q-values (one per
    operator).
    """
    def __init__(self, state_dim=STATE_DIM, n_actions=N_ACTIONS,
                 hidden=64):
        super().__init__()
        self.fc1 = nn.Linear(state_dim, hidden)
        self.fc2 = nn.Linear(hidden, hidden)
        self.out = nn.Linear(hidden, n_actions)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.out(x)
```

A.2 DQN MANAGER — OPERATOR SELECTION

Listing A.2 — DQN Manager: action selection, replay buffer, and Bellman update.

```
class DQNManager:
    def __init__(self, state_dim=STATE_DIM, n_actions=N_ACTIONS,
                 lr=1e-3, gamma=0.9,
                 eps_start=1.0, eps_min=0.1, eps_decay=0.97):
        self.gamma = gamma
        self.eps = eps_start
        self.eps_min = eps_min
        self.eps_decay = eps_decay
        self.buffer = [] # capacity 1000
        self.policy = DQNNet(state_dim, n_actions)
        self.target = DQNNet(state_dim, n_actions)
```

```

        self.target.load_state_dict(self.policy.state_dict())
        self.opt =
torch.optim.Adam(self.policy.parameters(), lr=lr)

def pick_action(self, state):
    """Epsilon-greedy operator selection."""
    if random.random() < self.eps:
        return random.randint(0, N_ACTIONS - 1)
    with torch.no_grad():
        q =
self.policy(torch.from_numpy(state.astype(np.float32)))
    return int(q.argmax().item())

def remember(self, s, a, r, s2):
    self.buffer.append((s, a, r, s2))
    if len(self.buffer) > 1000:
        self.buffer.pop(0)

def train_step(self, batch_size=16):
    if len(self.buffer) < batch_size:
        return
    batch = random.sample(self.buffer, batch_size)
    states, acts, rews, nexts = zip(*batch)
    S = torch.FloatTensor(states)
    A = torch.LongTensor(acts).unsqueeze(1)
    R = torch.FloatTensor(rews)
    S2 = torch.FloatTensor(nexts)
    q_pred = self.policy(S).gather(1, A).squeeze()
    with torch.no_grad():
        q_next = self.target(S2).max(1)[0]
    target = R + self.gamma * q_next
    loss = F.mse_loss(q_pred, target)
    self.opt.zero_grad(); loss.backward(); self.opt.step()
    self.eps = max(self.eps_min, self.eps * self.eps_decay)

```

A.3 OPERATOR EXECUTOR — 7-ACTION DISPATCH

Listing A.3 — Operator Executor: 7-action dispatch.

```

def apply_operator(self, action, ind, t_ratio, leader, mate,
cur_obj):
    """Dispatch the action chosen by the DQN to the matching
operator."""
    if action == 0: return self._hbo_dig(ind, t_ratio), False
    if action == 1: return self._hbo_honey(ind, leader), False
    if action == 2: return self._hbo_sniff(ind), False
    if action == 3: return self._hbo_jump(ind), False
    if action == 4: return self._hbo_swap(ind), False
    if action == 5: return self._crossover(ind, mate), False
    if action == 6:
        # Returning True signals the outer loop to run memetic
local
        # search on this individual.
        return ind.copy(), True
    raise ValueError(f"unknown action {action}")

```

A.4 HBO MOVE IMPLEMENTATIONS

Listing A.4 — HBO moves with iteration-decayed parameters (no α/β).

```
def _hbo_dig(self, ind, t_ratio):
    """Coarse exploit. Both step and n_mut decay with
    iteration."""
    intensity = 0.55 * (1 - t_ratio) + 0.05      # 0.60 -> 0.05
    step      = max(1, int(self.num_procs * intensity * 0.25))
    n_mut     = max(1, int(self.num_tasks * (0.5*(1-t_ratio) +
    0.05)))
    new       = ind.copy()
    idxs      = np.random.choice(self.num_tasks, n_mut,
    replace=False)
    new[idxs] = np.clip(ind[idxs] + np.random.randint(-step,
    step+1, n_mut),
                        0, self.num_procs - 1)
    return new

def _hbo_honey(self, ind, leader):
    """Fine exploit. Copy 50% of genes from current leader."""
    new       = ind.copy()
    mask      = np.random.rand(self.num_tasks) < 0.5
    new[mask] = leader[mask]
    return new

def _hbo_sniff(self, ind):
    """Explore. Random reassignment of 10% of tasks."""
    new       = ind.copy()
    k         = max(1, int(self.num_tasks * 0.1))
    idxs      = np.random.choice(self.num_tasks, k, replace=False)
    new[idxs] = np.random.randint(0, self.num_procs, k)
    return new

def _hbo_jump(self, ind):
    """Escape. Full random restart of this individual."""
    return np.random.randint(0, self.num_procs, self.num_tasks)

def _hbo_swap(self, ind):
    """Mutation. Swap one pair of task-to-processor
    assignments."""
    new = ind.copy()
    if self.num_tasks >= 2:
        a, b = np.random.choice(self.num_tasks, 2,
    replace=False)
        new[a], new[b] = new[b], new[a]
    return new
```

A.5 REWARD SHAPING

Listing A.5 — Reward shaping used by the DQN Manager.

```
def compute_reward(prev_best, new_best, prev_spread, new_spread):
    """
     $\Delta$ (best score) rewards operators that improve the best
    individual.
```

```

    A small 0.05 coefficient on  $\Delta(\text{spread})$  keeps the Pareto front
from
    collapsing without dominating the primary signal.
    """
    return float(prev_best - new_best) + 0.05 * (new_spread -
prev_spread)

```

A.6 MAIN SCHEDULING LOOP (ABRIDGED)

Listing A.6 — Main scheduling loop tying the modules together.

```

def run_scheduler(dag, processor_pool, T=150, K=50):
    """
    Full DQN-Guided NSGA-II with Operator Executor.
    Returns the final Pareto archive A_T.
    """
    env = MultiprocessorEnvironment(dag, processor_pool)
    nsga2 = NSGA2Evaluator(n_objectives=3)
    dqn = DQNManager()
    ops = OperatorExecutor(num_tasks=len(dag.nodes),
                           num_procs=len(processor_pool))

    pop = init_random_population(K, len(processor_pool),
                                len(dag.nodes))
    for ind in pop: ind.objectives = env.evaluate(ind)
    archive = nsga2.sort(pop)

    prev_best = best_score(pop)
    prev_spread = pareto_spread(archive)

    for t in range(1, T + 1):
        state = build_state_vector(pop, archive, t, T, dqn)
        action = dqn.pick_action(state)
        leader = archive[0]
        mate = random.choice(pop)

        offspring = []
        for ind in pop:
            child, run_memetic = ops.apply_operator(
                action, ind, t/T, leader, mate, ind.objectives)
            if run_memetic:
                child = ops.memetic_step(child, env)
                child.objectives = env.evaluate(child)
                offspring.append(child)

        pop, archive = nsga2.select(pop + offspring, K)
        new_best = best_score(pop)
        new_spread = pareto_spread(archive)
        reward = compute_reward(prev_best, new_best,
                                prev_spread, new_spread)
        next_state = build_state_vector(pop, archive, t+1, T,
dqn)
        dqn.remember(state, action, reward, next_state)
        dqn.train_step(batch_size=16)
        prev_best, prev_spread = new_best, new_spread

```

return archive

APPENDIX B

SUPPLEMENTARY TABLES AND ANALYSIS

B.1 OPERATOR-USAGE ANALYSIS ACROSS 30 RUNS

Section 5.6 noted that the DQN learns to favour different operators at different stages of the run. Table B.1 reports the average fraction of generations in which each operator was selected by the greedy action, broken down into three windows: early (generations 1–50), mid (51–100), and late (101–150). The numbers are averaged across the 30 independent runs of Experiment 2 on `j_139493`.

Table B.1 — Average per-window operator-usage frequency across 30 runs of Experiment 2.

Operator	Early (gen 1–50)	Mid (gen 51–100)	Late (gen 101–150)
Dig	14 %	19 %	32 %
Honey	13 %	17 %	38 %
Sniff	16 %	24 %	5 %
Jump	15 %	19 %	3 %
Swap	14 %	9 %	6 %
Crossover	14 %	8 %	8 %
Memetic LS	14 %	4 %	8 %

Three patterns are visible. First, the early-window distribution is approximately uniform, reflecting the high ε (≈ 0.85 – 1.0) in that window — the DQN is mostly exploring rather than exploiting. Second, Sniff and Jump dominate the mid-window, when the policy has started to learn but the population still benefits from broad perturbation. Third, Honey and Dig dominate the late window, where the front is dense and fine-grained exploitation is what pushes the best individual further toward HEFT-optimal. Memetic LS appears more in the very late stage (the final 10–20 generations) as a finishing operator on the best individual.

B.2 PER-JOB ENERGY AND UTILISATION BREAKDOWN

Table B.2 reports the per-job energy consumption and processor utilisation of the proposed scheduler across the five Experiment 1 workloads, alongside the corresponding numbers for the strongest baseline on each job. The pattern is consistent: the proposed scheduler achieves higher utilisation, which translates directly into lower idle-power consumption.

Table B.2 — Per-job energy and utilisation: proposed vs best baseline.

Job	Tasks	Best baseline (J / Util)	Proposed (J / Util)	Energy Δ
j_2926852	100	GWO 73 600 J / 41.6 %	66 124 J / 65.5 %	−10.2 %
j_1787378	199	GWO 6 224 J / 7.9 %	7 050 J / 7.9 %	+13.3 %
j_139493	301	GWO 91 350 J / 59.4 %	84 806 J / 94.8 %	−7.2 %
j_3166034	400	GWO 550 062 J / 49.6 %	523 744 J / 62.7 %	−4.8 %
j_652317	492	PSO 122 127 J / 43.7 %	110 268 J / 75.2 %	−9.7 %

The single exception is j_1787378, where the proposed scheduler uses approximately 13 per cent more energy than GWO. Both methods reach the same makespan (within 0.1 seconds of HEFT-optimal), but the proposed scheduler's slightly higher peak-power utilisation pattern carries a small energy overhead. This is the only workload in the sweep where energy is not improved, and it matches the observation that on the smallest workload the proposed scheduler offers no advantage over a well-tuned GA.

B.3 CELL 12 CONSOLE LOG EXCERPT

The following is a verbatim excerpt from the Cell 12 console output that produced Tables 5.4 and 5.5. The excerpt covers the $P = 200$ sweep, where the gap between the proposed scheduler and the baselines is largest.

Listing B.1 — Verbatim Cell 12 console output ($P = 200$ + final summary table).

```
=====
=====
EXPERIMENT 5: PROCESSOR-COUNT SCALABILITY (fixed job = j_139493)
  P sweep = [10, 25, 50, 100, 200] | Runs=3 | MaxIter=150 | Pop=50
| Obj=3
```



```
=====
=====
Fixed job: j_139493 | Tasks = 301
```

```
-----
-----
P = 200 processors
```

```
-----
HEFT=484.5s | SLA=726.8s (HEFT x 1.5)
GA ... Makespan=952.6+/-2.0s |
EXCEEDS SLA | DMR=31.1% | Energy=101807J | Util=37.0%
PSO ... Makespan=809.4+/-88.5s |
EXCEEDS SLA | DMR=11.4% | Energy=97490J | Util=44.0%
GWO ... Makespan=739.9+/-12.8s |
EXCEEDS SLA | DMR=2.0% | Energy=95425J | Util=47.6%
EHO ... Makespan=944.2+/-3.9s |
EXCEEDS SLA | DMR=29.9% | Energy=101508J | Util=37.3%
HBO ... Makespan=880.1+/-91.7s |
EXCEEDS SLA | DMR=21.1% | Energy=99600J | Util=40.5%
RL ... Makespan=951.2+/-0.0s |
EXCEEDS SLA | DMR=30.9% | Energy=101737J | Util=37.0%
Proposed(DQN+NSGA2+HBO) ... Makespan=484.5+/-0.0s |
BEATS SLA | DMR=0.0% | Energy=87826J | Util=72.7%
```

```
=====
=====
FINAL RESULTS - Job j_139493 (301 tasks) under varying P
```

P	HEFT(s)	SLA(s)	GA	PSO	GWO
EHO	HBO	RL	Proposed		
10	7065.1	10597.6	7463.3	7169.6	7409.2
7161.9	7273.1	7456.4	7060.9		
25	2853.6	4280.4	3447.2	3069.5	3252.6
3159.4	3292.5	3511.1	2849.8		
50	1441.1	2161.6	2048.2	1884.4	1905.2
1876.2	1971.8	2092.2	1438.3		
100	745.2	1117.9	1301.1	1183.1	1181.7
1211.9	1269.3	1352.7	742.5		
200	484.5	726.8	952.6	809.4	739.9
944.2	880.1	951.2	484.5		

```
-----
-----
Saved: exp5_processor_scalability.png
```

B.4 CONFUSION BETWEEN ENERGY AND UTILISATION — A NOTE

A subtle point worth addressing here. A reader looking at Table 5.4 and Table B.2 together may expect that the algorithm with the lowest energy consumption also has the lowest utilisation, on the reasoning that fewer joules are spent if processors are turned off. The actual data shows the opposite: the proposed scheduler has both the lowest energy and the highest utilisation. This is because the energy model is dominated by idle-power consumption integrated over the makespan window. A schedule that finishes in half the wall-clock time saves more energy through shortened idle-power integration than it costs through higher peak-power activity. Utilisation and energy are therefore positively correlated in this regime, and improving one improves the other.

This correlation breaks down only at very small workloads (e.g. j_1787378 in Table B.2), where the makespan is already so short that idle-power integration is no longer dominant. In that regime the small peak-power overhead of the proposed scheduler shows up as a slight energy increase. The crossover point between the two regimes is approximately 250 tasks for the workloads we tested.

B.5 SENSITIVITY SKETCH — WHY $1.5 \times$ HEFT FOR THE SLA?

The choice of $1.5 \times$ HEFT as the soft SLA threshold deserves a short justification. Three considerations went into the choice.

First, prior work in the Alibaba-trace literature (Manasrah and Ali, 2018; Mao et al., 2019) uses multipliers in the range 1.2 to 2.0, with 1.5 being a common middle ground. Adopting 1.5 allowed direct comparison with that prior work without forcing the metaheuristics into either an unrealistically tight or unrealistically loose deadline regime.

Second, on the five workloads tested, $1.5 \times$ HEFT produces a meaningful separation between methods. At $1.2 \times$ HEFT, every method except the proposed scheduler misses the SLA on all five workloads, which collapses the comparison; at

$2.0 \times$ HEFT, most methods meet the SLA on most workloads, which also collapses the comparison. The $1.5 \times$ HEFT setting puts the SLA in the middle of the operating envelope where differences between methods are visible.

Third, the soft SLA is a diagnostic metric only — it does not enter the optimisation objective directly. The proposed scheduler optimises makespan, energy, and waiting time in the Pareto sense, and we report DMR alongside as a derived metric. Changing the SLA multiplier would change the DMR numbers reported in the tables but would not change the underlying schedules produced. In that sense the SLA choice is a reporting convention, not a design parameter.